Roadmap Backend .NET → Cloud

Guida di studio completa e autosufficiente — da sviluppatore junior a backend .NET solido e assumibile sul mercato internazionale.
Versione 3.2 · agosto 2026 · 20 sezioni + appendice · 238 h di nucleo, 323 h complete · 4–8 mesi a 10 ore/settimana

> **Se hai solo quattro mesi — il percorso minimo.**
>
> Questo documento è lungo, e non va preso o lasciato in blocco. Ogni sezione porta un'etichetta di priorità, e il **nucleo** da solo è un percorso completo e difendibile.
>
> **Fino alla candidabilità servono 153 ore di solo nucleo** — sezioni 0, 1, 2, 3, 3bis, 4, 5, 5bis — più i tre artefatti obbligatori: README presentabile, deploy pubblico raggiungibile via URL, Swagger navigabile. Sono circa **quattro mesi a 10 ore/settimana**. Da lì ci si candida, e tutto il resto si studia mentre le candidature girano.
>
> Quando sarai in ritardo — e succederà — la domanda giusta non è "mollo?" ma "quali comprimibili taglio?". Ogni sezione comprimibile dichiara esplicitamente cosa perdi saltandola, così la scelta è informata e non te la rimangi la settimana dopo.

Le tre priorità

Etichetta	Ore	Significato
NUCLEO	**238**	Non si salta. È il profilo che il mercato dà per assodato in un backend .NET. Saltarne una parte lascia un buco che emerge a colloquio.
CONSIGLIATO	+62	Sposta la probabilità in modo misurabile e produce materiale da raccontare a colloquio. Da fare, ma dopo il nucleo e in parallelo alle candidature.
COMPRIMIBILE	+23	Valore reale ma marginale rispetto al costo. Prima cosa da tagliare se il tempo stringe, o da rimandare a dopo l'assunzione.

Somma completa: 323 ore. Le stime sono state riviste al rialzo dove la versione precedente era ottimistica: vale più un numero scomodo e vero che uno comodo e sbagliato.

Parte I — Perché questo documento esiste

Il punto di partenza

Il destinatario di questa roadmap è uno sviluppatore con circa un anno di esperienza professionale in C#/.NET, ASP.NET Core, JavaScript e SQL (SQL Server e SQL Anywhere), che lavora su software gestionale e su driver di integrazione tra macchinari industriali ed ERP/MES in ambito Industria 4.0. Laurea triennale in informatica. Madrelingua italiano e inglese.

Il problema che questo documento risolve non è la mancanza di lavoro — un lavoro c'è già. È un altro, ed è duplice:

1. **Una competenza percepita come incompleta.** Molto del lavoro quotidiano avviene con l'assistenza dell'AI: si descrive l'approccio, l'AI traduce in codice. Il risultato funziona, ma la comprensione si ferma al 60-70%, e resta il dubbio fondato di non saper riscrivere quel codice da zero. Questo è un debito che si paga al primo colloquio tecnico serio.
2. **Un profilo non ancora spendibile sul mercato che interessa.** Le competenze ci sono in forma grezza, ma mancano gli elementi che il mercato dichiara esplicitamente di volere.

Gli obiettivi

- **Primo e immediato:** diventare solidi in ciò che si fa già. Sapere davvero, non cavarsela.
- **Secondo:** poter cambiare lavoro con alta probabilità di successo, verso posizioni meglio retribuite, in mercati anglofoni (non è prevista l'acquisizione di altre lingue).
- **Terzo:** costruire competenze *trasversali*, valide indipendentemente da geografia e datore di lavoro, che tengano aperte le evoluzioni successive (DevOps, cloud, sicurezza) senza dover ricominciare.

Il target geografico prioritario è oggi l'Austria (Vienna in particolare, in presenza o da remoto) oppure una posizione full remote internazionale. Questo dettaglio è però secondario e potrà cambiare: la roadmap è costruita su competenze richieste dal mercato globale, non su un singolo paese.

Su cosa si basa questa roadmap

Fonte primaria: il profilo standard del backend developer .NET. Questo profilo è sorprendentemente stabile e uniforme. Le principali roadmap pubbliche di riferimento (roadmap.sh, le roadmap community per ASP.NET Core, le guide dei principali autori dell'ecosistema) concordano quasi parola per parola su cosa un backend .NET deve sapere, e le grandi rilevazioni sull'uso reale degli strumenti — come la State of Developer Ecosystem di JetBrains, *basata su 24.534 sviluppatori in 194 paesi nell'edizione*

2025 — confermano quali tecnologie siano effettivamente diffuse e quali siano solo rumore.

Il consenso è questo, ed è la spina dorsale del documento:

> **Nucleo irrinunciabile:** C# e i suoi meccanismi profondi · OOP e principi di design · SQL e database relazionali · ASP.NET Core (API, routing, middleware, dependency injection, autenticazione) · un ORM, in pratica Entity Framework Core · Git · testing automatizzato.
>
> **Standard di fatto nel 2026:** Docker come unità di deployment · una pipeline CI/CD · almeno un cloud, e per l'ecosistema Microsoft è Azure · logging strutturato e osservabilità · caching · messaggistica asincrona e resilienza. Sono passati da "nice to have" a requisito di base nel giro di pochi anni.
>
> **Specializzazioni successive, non ora:** Kubernetes e infrastructure as code · front-end · architetture distribuite avanzate · sicurezza applicativa e cloud security.

Fonte secondaria: una verifica sul campo. Nell'agosto 2026 sono stati esaminati una trentina di annunci reali su più categorie di ruolo e più mercati. Va detto chiaramente che si tratta di un campione piccolo e scelto a mano, quindi *non statisticamente rappresentativo*: non è la base della roadmap, ma un controllo di realtà. Ha confermato il nucleo sopra e ha aggiunto due osservazioni di contesto utili:

- **Il filtro più frequente non è la tecnologia, sono gli anni dichiarati.** Molti annunci chiedono 3-5 anni anche quando lo stack richiesto è alla portata. Due canali fanno eccezione e vanno tenuti presenti: le grandi società di consulenza, che assumono junior a volume e formano, e il mercato austriaco in presenza, dove numerosi annunci accolgono esplicitamente chi è alla prima esperienza.
- **Il mercato non chiede di rinunciare all'AI, chiede di dominarla.** Alcuni annunci richiedono esplicitamente la capacità di usare gli LLM nel ciclo di sviluppo con giudizio critico dimostrabile, e i processi di colloquio si stanno adattando: prove progettate per tollerare Copilot, seguite da una discussione in cui il candidato deve difendere le proprie scelte. Chi ha solo fatto pattern matching emerge subito. Questo è esattamente il rischio che questa roadmap serve a neutralizzare.

Perché l'ecosistema Microsoft/Azure e non altro

È l'unica scelta che soddisfa contemporaneamente tutti gli obiettivi, e per questo è una decisione su cui non serve tornare:

- Capitalizza l'esperienza già maturata invece di azzerarla.
- È dominante nell'enterprise europeo e in modo particolare nell'area di lingua tedesca.
- È la porta d'ingresso naturale al DevOps (Azure DevOps, pipeline, infrastructure as code) e da lì al DevSecOps.
- È indipendente da lingua e geografia: Azure funziona allo stesso modo ovunque.

Cosa questo documento NON copre

Per scelta di priorità. Prima si diventa solidi sul backend .NET, poi si specializza.

- **Front-end** (Angular, React, Blazor). Escluso dal focus attuale. Blazor WebAssembly resta un'estensione naturale futura, essendo C#.
- **DevOps avanzato** (Kubernetes, Terraform, IaC, observability). Merita una roadmap dedicata, da affrontare dopo questa.
- **Cloud security e DevSecOps.** Idem: roadmap separata, successiva.
- **Certificazioni di sicurezza** (CompTIA Security+ e simili). Rinviate consapevolmente: le stesse ore rendono molto di più investite qui.

Parte II — Come usare questa guida

Metodo

- Le sezioni vanno affrontate **in ordine**: ognuna presuppone la precedente.
- Ogni sezione ha un **criterio di uscita** verificabile. Se non lo soddisfi, non passare avanti.
- Ogni sezione elenca **domande tipiche da colloquio**. Se sai rispondere a tutte senza cercare, sei pronto su quel tema.
- **Studio ed esercizio vanno in parallelo**, mai in sequenza. Ogni sezione produce codice.
- Le ore stimate presuppongono un anno di esperienza pratica pregressa. Se una sezione costa il doppio, era una lacuna vera e il tempo è ben speso.

> **Anticipo obbligatorio a fine Modulo B.** Alla chiusura del Modulo B — cioè alla soglia di candidabilità, a circa 157 ore — vanno già fatte tre cose che il documento colloca formalmente nella Sezione 12:
>
> 1. Un **README presentabile** di MachineHub: problema, stack, come si avvia, stato del lavoro.
> 2. Un **deploy minimo pubblico**, anche il più semplice possibile (App Service tier gratuito, o qualunque hosting raggiungibile via URL).
> 3. **Swagger** attivo e navigabile sull'ambiente pubblicato.
>
> Motivo: da fine Modulo B ci si candida. Senza un link da mettere nel CV si passano mesi a candidarsi senza avere nulla da mostrare, e questo è l'errore che vanifica metà del lavoro. La Sezione 12 rifinirà e completerà questi tre artefatti, non li creerà da zero.

Uso con un assistente AI

Il metodo previsto è una **chat dedicata per ogni sezione**. L'assistente non ha memoria tra le chat: questo documento è l'unico contesto.

> **Istruzioni per l'assistente AI.** Questa è una roadmap di studio. L'utente ti fornisce il documento e dichiara a che punto si trova.
>
> - Spiega i concetti in profondità, con esempi commentati. Il documento elenca gli argomenti ma *non li spiega*: quello è il tuo compito.
> - **Non scrivere il codice degli esercizi al posto dell'utente.** Fornisci struttura, indicazioni, revisione critica. L'utente sta deliberatamente riducendo la propria dipendenza dall'AI nella scrittura del codice: assecondare quella richiesta è parte del tuo compito.
> - Quando presenta codice, chiedi *perché* ha fatto una scelta prima di approvarla.
> - Prima di dichiarare completata una sezione, interrogalo sulle domande da colloquio elencate.
> - Segnala errori concettuali in modo diretto. L'obiettivo è la solidità, non la rassicurazione.

PROMPT DA INCOLLARE A INIZIO DI OGNI CHAT

```
Sto seguendo la roadmap allegata (PDF). Leggi la Parte I e la
Parte II per il contesto e le istruzioni destinate a te.

Sezioni completate: [elenco]
Sezione attuale: [numero e titolo]

Guidami: spiegami i concetti elencati, propongimi esercizi
aggiuntivi se servono, revisiona il codice che scrivo io.
Non scrivere il codice degli esercizi al posto mio.
Alla fine verificami sulle domande da colloquio e sul
criterio di uscita della sezione.
```

Organizzazione del lavoro pratico

REPOSITORY A — `DOTNET-LAB`

Palestra personale. Una cartella numerata per sezione (`01-csharp/`, `02-oop/`, …), con micro-esercizi ed esperimenti. In ogni cartella un `NOTES.md` dove scrivi **con parole tue** cosa hai capito. Questo file è il tuo manuale di ripasso futuro ed è anche la cura contro la sensazione di non sapere niente: dopo sei mesi vedi nero su bianco quanto hai imparato.

REPOSITORY B — `MACHINEHUB`

Il progetto guida. Un'unica applicazione che cresce lungo tutta la roadmap, sezione dopo sezione. Alla fine è il pezzo da portfolio e da colloquio.

MachineHub — specifica. API per la raccolta e distribuzione di dati di produzione industriale.

Macchine simulate inviano letture di produzione (id macchina, timestamp, pezzi prodotti, pezzi scartati, stato, codice allarme). Il sistema le riceve, le valida, le persiste, le espone via API REST autenticata, e le inoltra a un sistema gestionale simulato. Gestisce anagrafica macchine, commesse di produzione, allarmi, e report di efficienza.

Perché questo progetto e non un altro: rispecchia il dominio in cui il destinatario lavora già. A colloquio non racconta un tutorial ma un problema che conosce dall'interno, e questo è il singolo fattore che più differenzia un portfolio. In più, richiede naturalmente ogni competenza della roadmap: modellazione dati, API, autenticazione, testing, container, code, deploy in cloud.

Regola non negoziabile su entrambi i repository. L'AI può scrivere codice che sapresti scrivere da solo — lì è un acceleratore legittimo. Non può scrivere codice che *non* sapresti scrivere: quello è un prestito che non stai restituendo. In pratica, durante lo studio la scelta di default resta digitare tu: se una riga non sapresti riscriverla da zero, la sezione non è finita. Vedi la Regola permanente 1 in fondo al documento.

Parte III — Quadro d'insieme

#	Sezione	Ore	Priorità	Modulo
0	Setup e fondamenta	3	NUCLEO	**A — Linguaggio** 61 h, tutte nucleo
1	C#: dalle basi alla profondità	38	NUCLEO	
2	OOP, design e architettura	20	NUCLEO	
3	SQL e database relazionali	30	NUCLEO	**B — Backend .NET** 96 h, di cui 92 nucleo **Fine del nucleo di B = candidabilità: 153 ore, circa 4 mesi**
3bis	HTTP e fondamenta del web	4	NUCLEO	
4	ASP.NET Core	30	NUCLEO	
5	Entity Framework Core	20	NUCLEO	
5bis	Sicurezza applicativa	8	NUCLEO	
5ter	Real-time con SignalR	4	COMPRIMIBILE	
6	Testing	15	NUCLEO	**C — Professionalità** 31 h, di cui 23 nucleo
6bis	Diagnostica e performance	6	CONSIGLIATO	
7	Git avanzato e collaborazione	8	NUCLEO	
7bis	Agile e Scrum	2	CONSIGLIATO	
8	Docker e container	15	NUCLEO	**D — Cloud e delivery** 98 h, di cui 62 nucleo
9	CI/CD	12	NUCLEO	
10	Azure	35	NUCLEO	
10bis	NoSQL, il minimo indispensabile	3	COMPRIMIBILE	
11	Messaggistica asincrona	12	CONSIGLIATO	
11bis A	Microservizi e CQRS — teoria	7	CONSIGLIATO	
11bis B	Microservizi e CQRS — laboratorio	14	COMPRIMIBILE	
12	Consolidamento, deploy, documentazione	35	CONSIGLIATO	**E — Prova**
—	Appendice: consapevolezza tecnologica	2	COMPRIMIBILE	Da leggere, non da studiare
Solo nucleo		**238**	~6 mesi — percorso completo e difendibile	
Nucleo + consigliato		**300**	~7,5 mesi — obiettivo realistico	
Tutto		**323**	~8 mesi	

La numerazione con suffisso (3bis, 5bis, 5ter…) è voluta: le sezioni sono state aggiunte in un secondo momento e mantenere invariati i numeri originali evita di invalidare appunti e tracker già compilati.

La numerazione con suffisso (3bis, 5bis, 5ter…) è voluta: le sezioni sono state aggiunte in un secondo momento e mantenere invariati i numeri originali evita di invalidare appunti, note e tracker già compilati.

Cosa si sblocca, modulo per modulo

Al termine di	Posizioni a cui candidarsi	Cosa puoi affermare in colloquio
Modulo A 61 h	Nessuna nuova: è il prerequisito. Ma smetti di temere le domande sul linguaggio.	Conosco il linguaggio, non solo la sintassi.
Modulo B 157 h totali ≈ 4 mesi	**Soglia di candidabilità.** Junior/Mid .NET Developer · Backend Developer C# · Software Engineer .NET · ERP Developer su piattaforme .NET · Application Developer · programmi Trainee e Graduate	So progettare e realizzare un'API sicura, con persistenza e funzionalità real-time, in autonomia. Ed eccola online.
Modulo C 188 h totali	Le stesse, con probabilità nettamente più alte. Testing e diagnostica sono i differenziatori più economici a questo livello, e lo Scrum toglie di mezzo un filtro banale ma reale.	So lavorare in team, garantire la qualità di ciò che consegno e diagnosticare un problema in produzione.
Modulo D 272 h totali	Aggiunge: Backend Developer cloud-native · Integration Developer · MES / Industrial Software Engineer · Junior DevOps / Platform Engineer · posizioni full remote internazionali · ruoli su architetture a microservizi	So portare in produzione ciò che scrivo, non solo scriverlo. E so ragionare su un sistema distribuito.

Modulo E 309 h totali	Tutte le precedenti, con un portfolio verificabile e tre racconti pronti.	Ecco un sistema che ho costruito. Chiedimi qualsiasi cosa.

> **Nota strategica sui tempi.** Non serve completare tutto prima di candidarsi. Il **Modulo B** è la soglia di candidabilità: da lì in poi si applica in parallelo allo studio. Prima di quel punto, candidature solo su occasioni ad altissimo match — non candidature di massa, che brucerebbero le opportunità migliori.

MODULO A — Linguaggio

Sezioni 0-2 · 61 ore, tutte nucleo · La base. Nessuna scorciatoia possibile.

Sezione 0 — Setup e fondamenta

3 ore · Modulo A · NUCLEO

OBIETTIVO

Eliminare ogni attrito operativo prima di iniziare, e definire il progetto per iscritto prima di scrivere codice.

ATTIVITÀ

1. Installare: .NET SDK (ultima LTS), un IDE (Visual Studio, Rider o VS Code con estensione C# Dev Kit), Docker Desktop, SQL Server Developer Edition o container, Azure Data Studio o SSMS, Git.
2. Creare i repository `dotnet-lab` e `machinehub` su GitHub, pubblici, con README.
3. Scrivere nel README di `machinehub` due paragrafi che descrivono il problema che l'applicazione risolve. *Perché:* chi non sa scrivere cosa fa un sistema non sa nemmeno costruirlo, e questo testo diventerà la base del README finale da portfolio.
4. Impostare `.gitignore` corretto per .NET e `.editorconfig` con le convenzioni di stile.
5. **Fissare esplicitamente la versione di .NET** del progetto (l'ultima LTS disponibile), dichiararla nel `global.json` e **scriverla nel README di MachineHub**. *Perché:* senza una versione fissata, sei mesi di lavoro attraversano due o tre release, le guide che troverai si riferiranno a versioni diverse, e passerai tempo a inseguire differenze che non c'entrano con quello che stai imparando.

> **Criterio di uscita:** entrambi i repository esistono, un progetto console compila e il primo commit è fatto, e sai spiegare a voce cosa farà MachineHub.

Sezione 1 — C#: dalle basi alla profondità

38 ore (8 di ripasso rapido + 30 di approfondimento) · Modulo A · NUCLEO

OBIETTIVO

Passare da "so scrivere C# che funziona" a "so perché funziona". È la sezione che colma direttamente il divario di competenza percepito. Si divide in due parti: un ripasso rapido delle fondamenta, e l'approfondimento vero.

Parte A — Ripasso rapido delle basi

8 ore · saltabile in tutto o in parte, a discrezione

> **Perché c'è questa parte.** Nessuna roadmap dovrebbe dare per scontate le basi, perché le lacune elementari sono quelle che fanno crollare un colloquio nel momento peggiore: non quando ti chiedono l'architettura, ma quando ti chiedono la differenza tra `throw` e `throw ex`, o perché due stringhe uguali possono risultare diverse. Sono argomenti che con un anno di esperienza probabilmente sai già: scorri l'elenco, marca quelli su cui hai il minimo dubbio, ripassa solo quelli. Ma **guardali tutti** almeno una volta, perché le lacune di cui sei consapevole non sono il problema.

A.1 — STRUTTURA E SINTASSI

- Struttura di un programma C#: namespace, `using`, classi, metodo `Main`, top-level statements.
- Convenzioni di naming (PascalCase, camelCase, prefisso `_` per i campi privati), commenti, XML documentation comment.
- Compilazione, assembly, IL e ruolo del runtime .NET in sintesi.
- Struttura di una solution e di un progetto, file `.csproj`, pacchetti NuGet.

A.2 — VARIABILI E TIPI

- Dichiarazione e inizializzazione; `var` contro tipo esplicito e quando l'uno o l'altro; `const` contro `readonly`.
- Tipi primitivi: `int`, `long`, `short`, `byte`, `float`, `double`, **`decimal`** (e perché per gli importi si usa `decimal` e mai `double`), `bool`, `char`, `string`, `DateTime`, `Guid`.
- Overflow, `checked` e `unchecked`; valori di default dei tipi.
- Conversioni: implicite, esplicite (cast), `Convert`, `Parse` contro **`TryParse`**.
- Scope e durata delle variabili.

A.3 — OPERATORI

- Aritmetici; divisione intera contro divisione decimale; modulo; incremento e decremento (prefisso contro postfisso).
- Confronto e uguaglianza: `==` contro `Equals`, e cosa cambia tra stringhe, tipi valore e riferimenti.
- Logici `&&`, `||`, `!` e valutazione short-circuit (e perché conta).

- Bitwise `&`, `|`, `^`, `<<`, `>>` e gli enum con `[Flags]`.
- Assegnazione composta; operatore ternario `?:`.
- Operatori null: `??`, `??=`, `?.`, `?[]`, e l'operatore null-forgiving `!`.
- `is`, `as`, `typeof`, `nameof`, `sizeof`.
- Precedenza degli operatori e quando mettere le parentesi anche se non servono.

A.4 — CONTROLLO DI FLUSSO

- `if` / `else if` / `else`; annidamento e early return come alternativa.
- `switch` **statement** (case, default, fall-through non consentito in C#) e `switch` **expression** moderna con pattern matching.
- Cicli: `for`, `foreach`, `while`, `do-while` — quando usare quale; cicli annidati.
- `break`, `continue`, `return`; `goto` e perché non si usa.
- `yield return` e cosa significa restituire pigramente una sequenza.

A.5 — GESTIONE DEGLI ERRORI

- `try` / `catch` / `finally`: cosa viene eseguito e in quale ordine.
- `catch` multipli e ordine dal più specifico al più generico; filtri con `when`.
- `throw` **contro** `throw ex`: la differenza sullo stack trace. Domanda da colloquio classica.
- Eccezioni comuni e cosa significano davvero: `NullReferenceException`, `IndexOutOfRangeException`, `ArgumentException`, `InvalidOperationException`, `FormatException`, `DivideByZeroException`.
- Eccezioni custom; `InnerException`.
- Perché le eccezioni non vanno usate per il normale flusso di controllo, e il pattern `TryParse` come alternativa.

A.6 — METODI

- Firma, parametri, valore di ritorno, `void`; overloading e come il compilatore sceglie.
- Parametri opzionali, argomenti nominati, `params`.
- `ref`, `out`, `in`.
- Metodi statici contro metodi di istanza; metodi di estensione.
- Expression-bodied member; funzioni locali.
- Ricorsione e i suoi limiti (stack overflow).

A.7 — CLASSI E OGGETTI, LIVELLO BASE

- Come si istanzia un oggetto (`new`); riferimento contro oggetto; `null`.
- Campi, proprietà (auto-property, `get` / `set`, `init`, proprietà calcolate), costanti.
- Costruttori, costruttore di default, overloading dei costruttori, concatenamento con `: this()`.
- Modificatori di accesso: `public`, `private`, `protected`, `internal`, `protected internal`.
- `static` (campi, metodi, classi), `this`, `sealed`, `partial`.
- Object initializer; `enum`; `struct` di base; classi annidate.
- `ToString`, `Equals`, `GetHashCode`: esistenza e a cosa servono (approfonditi nella Parte B).

A.8 — STRINGHE E ARRAY

- **Immutabilità delle stringhe** e perché concatenare in un ciclo è un problema; `StringBuilder`.
- Interpolazione `$"..."`, verbatim `@"..."`, raw string literal.
- Metodi comuni: `Split`, `Join`, `Substring`, `Replace`, `Trim`, `Contains`, `StartsWith`, `IndexOf`.
- Confronto di stringhe e cultura (`StringComparison`), maiuscole e minuscole; formattazione di numeri e date.
- Array mono e multidimensionali, array frastagliati (jagged); indici e range (`^` e `..`).

A.9 — CONCETTI GENERALI DI PROGRAMMAZIONE

- Tipizzazione statica e forte: cosa comporta rispetto a un linguaggio dinamico.
- Stack e heap: introduzione (approfondita nella Parte B).
- Passaggio per valore e per riferimento: introduzione.
- **Complessità computazionale** e notazione O grande, almeno per O(1), O(log n), O(n), O(n log n), O(n^2). Serve a scegliere la struttura dati giusta e compare nei colloqui.
- Debugging: breakpoint, breakpoint condizionali, watch, call stack, step into/over/out, immediate window. Saper usare il debugger è una competenza sottovalutata e sostituisce ore di tentativi.

ESERCIZI DI RIPASSO (RAPIDI)

1. **Uguaglianza.** Costruisci casi in cui `==` e `Equals` danno risultati diversi: su stringhe, su tipi valore, su oggetti. *Obiettivo:* è la fonte di bug elementari più comune.
2. **throw contro throw ex.** Scrivi due catch identici, uno con `throw` e uno con `throw ex`, e confronta lo stack trace. *Obiettivo:* vedere la differenza invece di ricordarla.
3. **Concatenazione.** Concatena 100.000 stringhe in un ciclo, prima con `+` poi con `StringBuilder`, e misura. *Obiettivo:* capire l'immutabilità attraverso il suo costo.
4. **decimal contro double.** Somma `0.1` dieci volte con entrambi e osserva il risultato. *Obiettivo:* capire perché gli importi non si

trattano con i floating point.

5. **Debugger.** Prendi un metodo con un bug e trovalo usando solo il debugger, senza aggiungere un solo `Console.WriteLine`. *Obiettivo:* acquisire un'abitudine che ti farà risparmiare anni.

DOMANDE DA COLLOQUIO SULLE BASI

- Differenza tra `throw` e `throw ex`.
- Differenza tra `const` e `readonly`.
- Quando usi `decimal` invece di `double`?
- `==` ed `Equals` sono la stessa cosa?
- Perché le stringhe sono immutabili e quali conseguenze ha?
- Differenza tra `Parse` e `TryParse`.
- Cosa fa il blocco `finally` se nel `try` c'è un `return`?
- Differenza tra `out` e `ref`.

> **Criterio di uscita della Parte A:** hai scorso tutti e nove i blocchi, hai ripassato quelli su cui avevi dubbi, e rispondi alle otto domande sopra senza esitare.

Parte B — C# in profondità

30 ore

CONCETTI TEORICI DA PADRONEGGIARE

- **Tipi e memoria:** value type e reference type; stack e heap; boxing e unboxing; `struct` contro `class`; passaggio per valore e per riferimento; `ref`, `out`, `in`; immutabilità; `record` e `record struct`.
- **Nullability:** nullable value type e nullable reference type; operatori `?.`, `??`, `??=`, `!`; contesto nullable abilitato e cosa segnala il compilatore.
- **Collezioni e complessità:** `List`, `Dictionary`, `HashSet`, `Queue`, `Stack`, `IEnumerable`, `ICollection`, `IReadOnlyList`; complessità computazionale delle operazioni principali; `GetHashCode` e `Equals` e perché vanno insieme.
- **LINQ:** sintassi metodo e sintassi query; esecuzione differita; `IEnumerable` contro `IQueryable`; materializzazione; operatori di proiezione, filtro, aggregazione, raggruppamento, join; insidie della doppia enumerazione.
- **Asincronia:** cosa fa realmente `async/await`; la state machine generata dal compilatore; `Task` contro `Thread`; thread pool; `Task.WhenAll` e `WhenAny`; deadlock da `.Result` e `.Wait()`; `ConfigureAwait`; `CancellationToken`; `IAsyncEnumerable`; async void e perché evitarlo.
- **Eccezioni e risorse:** gerarchia delle eccezioni; quando catturare e quando lasciar propagare; eccezioni custom; `finally`; `IDisposable` e `IAsyncDisposable`; `using`; garbage collector in sintesi e ruolo dei finalizzatori.
- **Generics:** tipi e metodi generici; vincoli; covarianza e controvarianza in sintesi.
- **Delegate, eventi, espressioni lambda, closure** e cosa cattura una closure.
- **C# moderno:** pattern matching, switch expression, tuple, deconstruction, target-typed new, top-level statements.

ESERCIZI DI APPROFONDIMENTO

1. **Laboratorio memoria.** Scrivi una `class` e una `struct` con gli stessi campi, passale a metodi che le modificano, e verifica sperimentalmente cosa cambia. *Obiettivo:* vedere con i tuoi occhi la differenza invece di ricordarla a memoria.
2. **Laboratorio LINQ.** Costruisci una catena di query su una collezione di almeno 10.000 elementi e determina, inserendo dei log, in quale riga esatta la query viene eseguita. Poi provoca una doppia enumerazione e misurane il costo. *Obiettivo:* interiorizzare l'esecuzione differita, che è la fonte più comune di bug di performance con LINQ.
3. **Deadlock deliberato.** Scrivi codice che va in deadlock usando `.Result`, osservalo bloccarsi, poi correggilo. *Obiettivo:* capire l'asincronia dal lato del fallimento, che è il modo in cui la si capisce davvero.
4. **Cancellazione.** Implementa un'operazione lunga che risponde correttamente a un `CancellationToken`. *Obiettivo:* è richiesto in ogni codebase seria e quasi nessun junior lo sa fare.
5. **IDisposable.** Implementa una classe che incapsula una risorsa non gestita e la rilascia correttamente. *Obiettivo:* collegare linguaggio e gestione delle risorse.
6. **Equals e GetHashCode.** Crea una classe usata come chiave di `Dictionary` e osserva cosa succede se implementi male `GetHashCode`. *Obiettivo:* capire perché il lookup è O(1) e quando smette di esserlo.

INCREMENTO SUL PROGETTO

Applicazione console che genera letture di macchina simulate, le mantiene in memoria e ne calcola statistiche di produzione con LINQ (pezzi/ora, tasso di scarto, tempo di fermo per macchina). Generazione asincrona con possibilità di cancellazione.

DOMANDE TIPICHE DA COLLOQUIO

- Qual è la differenza tra `IEnumerable` e `IQueryable`? Quando conta davvero?
- Cosa fa `await` al thread corrente? `async` crea un nuovo thread?
- Perché `.Result` può causare un deadlock e in quali contesti?
- Differenza tra `struct` e `class`, e quando useresti una `struct`?
- Cos'è il boxing e perché costa?

- Perché un `Dictionary` ha lookup O(1) e cosa lo degrada?
- Cosa succede se modifichi una collezione mentre la stai iterando?
- Quando implementi `IDisposable`?

> **Criterio di uscita della sezione:** rispondi a tutte le domande della Parte A e della Parte B senza consultare nulla, e hai svolto tutti gli esercizi scrivendo il codice a mano.

Sezione 2 — OOP, design e architettura

20 ore · Modulo A · NUCLEO

OBIETTIVO

Saper strutturare un'applicazione, non solo farla funzionare. È la differenza più visibile tra un profilo junior e uno mid, e la parte teorica che più frequentemente viene chiesta a colloquio.

CONCETTI TEORICI DA PADRONEGGIARE

- **Pilastri OOP** ripassati sul serio: incapsulamento, ereditarietà, polimorfismo, astrazione — ma soprattutto *quando* ciascuno è la scelta giusta.
- **Interfacce contro classi astratte:** criteri di scelta; interfacce multiple; default interface methods.
- **Composizione contro ereditarietà** e perché le gerarchie profonde sono quasi sempre un errore.
- **SOLID applicato:** per ciascuno dei cinque principi, saper riconoscere una violazione in codice reale e correggerla. Non le definizioni a memoria.
- **Altri principi:** DRY, KISS, YAGNI, Law of Demeter, e i loro limiti (l'astrazione prematura è un costo).
- **Design pattern** effettivamente usati in .NET: Repository, Unit of Work, Factory, Strategy, Options, Decorator, Adapter, Result/Either. Per ognuno: quale problema risolve, e quando è sovraingegneria.
- **Architettura a strati:** Presentation → Application → Domain → Infrastructure; direzione delle dipendenze; dependency inversion; cenni a Clean Architecture e Onion Architecture.
- **Modellazione del dominio:** entità, value object, aggregati; anemic domain model e perché è discusso; dove va la logica di business (non nel controller).
- **Cenni a CQRS** e al pattern Mediator (MediatR), che compaiono spesso negli annunci.

ESERCIZI

1. **Caccia alla violazione.** Prendi una classe reale dal tuo lavoro o da un progetto vecchio che fa almeno tre cose diverse, e separala motivando ogni confine. *Obiettivo:* allenare il riconoscimento, che è l'abilità vera; le definizioni si imparano in un'ora.
2. **Ereditarietà contro composizione.** Modella lo stesso problema (tipi diversi di macchina con comportamenti parzialmente comuni) prima con una gerarchia di ereditarietà, poi con la composizione. Confronta cosa succede quando aggiungi un requisito nuovo. *Obiettivo:* capire il costo dell'ereditarietà provandolo, non leggendolo.
3. **Pattern applicati.** Implementa Strategy per il calcolo dell'efficienza con formule diverse, e Factory per la creazione dei parser di formati di lettura diversi. *Obiettivo:* conoscere i pattern che userai davvero.
4. **Disegno dell'architettura.** Disegna su carta gli strati di MachineHub e le dipendenze tra loro, poi giustifica ogni freccia. *Obiettivo:* saper spiegare un'architettura a voce è una domanda da colloquio ricorrente.

INCREMENTO SUL PROGETTO

Ristrutturare la console app in una solution a più progetti: `MachineHub.Domain` (entità e logica di dominio, senza dipendenze esterne), `MachineHub.Application` (servizi e casi d'uso), `MachineHub.Infrastructure` (persistenza, per ora in memoria), `MachineHub.Console`. Le dipendenze devono puntare verso l'interno.

DOMANDE TIPICHE DA COLLOQUIO

• Spiegami i principi SOLID con un esempio concreto di violazione per ciascuno.
• Quando usi un'interfaccia e quando una classe astratta?
• Perché si dice "composizione invece di ereditarietà"?
• Cos'è la dependency inversion e in cosa differisce dalla dependency injection?
• Dove metteresti la logica di business in un'applicazione a strati, e perché non nel controller?
• Cos'è il pattern Repository e quali sono le critiche che gli si muovono?
• Quando un design pattern diventa sovraingegneria?

> **Criterio di uscita:** il progetto ha strati separati con dipendenze corrette, e sai spiegare in due minuti perché il progetto Domain non referenzia nulla.

MODULO B — Backend .NET

Sezioni 3 → 5ter · 96 ore, di cui 92 di nucleo · La soglia di candidabilità cade alla fine del nucleo di questo modulo: 153 ore totali. Al termine, README, deploy pubblico e Swagger sono obbligatori.

Sezione 3 — SQL e database relazionali

30 ore · Modulo B · NUCLEO

OBIETTIVO

È l'area di vantaggio: si fa già ogni giorno. Qui la si trasforma da pratica in competenza dichiarabile e difendibile. Compare come requisito in praticamente ogni annuncio backend analizzato.

CONCETTI TEORICI DA PADRONEGGIARE

- **Modellazione:** chiavi primarie e esterne, cardinalità delle relazioni, normalizzazione fino alla terza forma normale, quando e perché denormalizzare, tipi di dato e loro impatto sullo spazio.
- **Query:** tutti i tipi di JOIN (inner, left, right, full, cross, self); subquery correlate e non; CTE ricorsive e non; window function (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, aggregazioni con `OVER`); `GROUP BY` e `HAVING`; set operator (`UNION`, `INTERSECT`, `EXCEPT`); `MERGE`.
- **Indici:** clustered contro non-clustered; indici compositi e ordine delle colonne; colonne `INCLUDE`; indici covering; indici filtrati; selettività; costo degli indici in scrittura; **perché un indice non viene usato** (funzioni applicate alla colonna, conversioni implicite di tipo, wildcard iniziale, bassa selettività, statistiche obsolete).
- **Piani di esecuzione:** come leggerli; table scan contro index seek contro index scan; key lookup; nested loops, merge join, hash join; stime di cardinalità sbagliate; operatori costosi.
- **Transazioni e concorrenza:** ACID; livelli di isolamento (read uncommitted, read committed, repeatable read, serializable, snapshot); dirty read, non-repeatable read, phantom read; lock e escalation; deadlock e come diagnosticarli.
- **Oggetti di database:** stored procedure, funzioni, trigger, viste, viste indicizzate — e quando ciascuno è appropriato.
- **Sicurezza:** SQL injection e query parametrizzate; principio del privilegio minimo.
- **Portabilità:** SQL Server è il riferimento nell'ecosistema Microsoft, ma **PostgreSQL** è molto diffuso nei progetti cloud-native e nel remoto internazionale. Conoscere le differenze principali (tipi, sequenze, sintassi di paginazione) costa poche ore e allarga il bacino di annunci compatibili.
- **Accesso dati senza ORM:** ADO.NET in sintesi, **Dapper** (quando conviene rispetto a EF), `SqlBulkCopy` per gli inserimenti massivi.
- **Pattern applicativi:** paginazione **keyset** (o "seek") contro `OFFSET/FETCH` e perché la seconda degrada sui grandi offset; **soft delete**; colonne di **audit**; tabelle temporali (system-versioned); colonne **JSON** e quando hanno senso in un relazionale.

ESERCIZI

1. **Schema completo a mano.** Progetta e crea in SQL puro lo schema di MachineHub (macchine, letture, commesse, allarmi, utenti, ruoli), con vincoli, chiavi e uno script di seed che genera almeno 500.000 letture. *Obiettivo:* senza volume di dati non si vedono i problemi di performance, ed è lì che sta l'apprendimento.
2. **Ottimizzazione misurata.** Scrivi una query lenta sul dataset grande, apri il piano di esecuzione, identifica il collo di bottiglia, aggiungi l'indice giusto, rimisura. Documenta il prima e il dopo. *Obiettivo:* è letteralmente l'esercizio che ti chiederanno di raccontare a colloquio.
3. **Indice inutilizzato.** Crea un indice e poi scrivi tre query che, per motivi diversi, non lo usano. Spiega ognuno dei tre motivi. *Obiettivo:* capire gli indici dal lato del fallimento.
4. **Window function.** Calcola la produzione cumulativa per macchina per turno, e la variazione rispetto alla lettura precedente. *Obiettivo:* le window function separano chi sa SQL da chi sa scrivere `SELECT`.
5. **Concorrenza.** Apri due sessioni e provoca deliberatamente un deadlock, poi osservalo nei log di sistema e risolvilo. *Obiettivo:* i deadlock in produzione sono inevitabili, saperli leggere no.
6. **Palestra di composizione — 30/40 query a difficoltà crescente.** Scrivile tutte da zero sul tuo dataset, senza copiare, partendo da `SELECT` con filtri e arrivando a query con CTE multiple, window function annidate e aggregazioni condizionali. **Le ultime dieci a cronometro**, come fossero un colloquio. *Obiettivo:* tutti gli altri esercizi di questa sezione sono diagnostici — ti insegnano a capire una query esistente. Questo è l'unico che ti allena a *comporne* una da zero sotto pressione, che è esattamente ciò che ti verrà chiesto in una prova tecnica.
7. **Dapper contro EF.** Riscrivi due delle query più complesse con Dapper e confronta righe di codice, leggibilità e tempi. *Obiettivo:* saper motivare la scelta dello strumento, non subirla.
8. **Paginazione keyset.** Implementa la paginazione con `OFFSET` e con keyset sullo stesso dataset grande, e misura la differenza alla pagina 1 e alla pagina 5.000. *Obiettivo:* è il classico problema che emerge solo in produzione.

INCREMENTO SUL PROGETTO

Schema relazionale completo con indici motivati, script di seed, e un file `docs/database.md` che documenta *perché* esiste ogni indice.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra indice clustered e non-clustered. Quanti clustered può avere una tabella?

- Ho una query lenta: come procedi?
- Perché un indice potrebbe non essere usato?
- Cosa sono i livelli di isolamento e cosa cambia tra read committed e serializable?
- Cos'è un deadlock e come lo si risolve?
- Differenza tra `WHERE` e `HAVING`; tra `UNION` e `UNION ALL`.
- Quando useresti una stored procedure invece di logica in applicazione?
- Come si previene una SQL injection?

> **Criterio di uscita:** sai mostrare un piano di esecuzione prima e dopo un'ottimizzazione fatta da te spiegando la differenza operatore per operatore, e sai comporre da zero una query con CTE e window function in meno di dieci minuti.

Sezione 3bis — HTTP e fondamenta del web

4 ore · Modulo B · `NUCLEO`

OBIETTIVO

La Sezione 4 costruisce API REST dando per scontato il protocollo su cui viaggiano. Questa sezione colma il presupposto. È corta, ma è la base concettuale di metà delle domande da colloquio sul web.

CONCETTI TEORICI DA PADRONEGGIARE

- **Anatomia di richiesta e risposta:** riga di stato, header, body; header più comuni (`Content-Type`, `Accept`, `Authorization`, `Cache-Control`, `ETag`, `User-Agent`, header custom).
- **Semantica dei metodi:** GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS; metodi *safe* e metodi **idempotenti**, e perché la distinzione conta per retry e cache.
- **Status code:** le cinque famiglie e i codici che userai davvero — 200, 201, 202, 204, 301, 302, 304, 400, 401, 403, 404, 405, 409, 415, 422, 429, 500, 502, 503. Differenza tra 401 e 403, tra 400 e 422, quando 409.
- **Evoluzione del protocollo:** HTTP/1.1 e il problema dell'head-of-line blocking, **HTTP/2** (multiplexing, compressione degli header), cenni a HTTP/3 e QUIC.
- **TLS e HTTPS:** handshake in sintesi, certificati e catena di fiducia, HSTS, perché "HTTPS ovunque" non è un dettaglio.
- **Sessione e identità:** HTTP è stateless; **cookie contro token**; attributi dei cookie (`HttpOnly`, `Secure`, `SameSite`); dove vive un token e quali rischi comporta ciascuna scelta.
- **CORS:** la same-origin policy, cosa è un'origine, la richiesta di **preflight** `OPTIONS`, gli header di risposta, e perché CORS è una protezione del browser e non del server.
- **Caching HTTP:** `Cache-Control`, `Expires`, **ETag** e richieste condizionali, `304 Not Modified`, cache pubbliche e private.
- **Content negotiation** e compressione (gzip, brotli).
- **Infrastruttura:** cosa fa un **reverse proxy** (nginx, IIS, Application Gateway), **Kestrel** e perché in produzione sta spesso dietro un proxy, header inoltrati (`X-Forwarded-For`), terminazione TLS, load balancer.

ESERCIZI

1. **Osservazione diretta.** Apri gli strumenti per sviluppatori del browser su un sito qualsiasi e leggi header di richiesta e risposta di cinque chiamate diverse, spiegandone ognuna. *Obiettivo:* il protocollo si capisce guardandolo, non leggendolo.
2. **Chiamate a mano.** Esegui le stesse chiamate con `curl` mostrando gli header (`-v`), poi ripetile con un token di autorizzazione. *Obiettivo:* saper interrogare un'API senza Postman è utile quanto sembra banale.
3. **Preflight in azione.** Provoca deliberatamente un errore CORS chiamando la tua API da una pagina su origine diversa, osserva la richiesta `OPTIONS` e risolvi lato server. *Obiettivo:* CORS è la fonte di frustrazione numero uno di chi inizia, e sparisce appena la capisci.
4. **ETag.** Implementa un endpoint con ETag e verifica il `304` con una seconda chiamata condizionale. *Obiettivo:* caching HTTP applicato, non teorico.

INCREMENTO SUL PROGETTO

Non ancora codice di prodotto: una raccolta di chiamate documentate a MachineHub (file `.http` o collezione esportata) versionata nel repository, che diventerà utile in tutte le sezioni successive.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra PUT e PATCH. Quali metodi sono idempotenti?
• Differenza tra 401 e 403.
• Cos'è CORS e quale problema risolve? Protegge il server o il browser?
• Cos'è una richiesta di preflight e quando parte?
• Cookie o token per autenticare una SPA? Vantaggi e rischi di ciascuno.
• A cosa serve un ETag?
• Cosa cambia tra HTTP/1.1 e HTTP/2?
• Perché Kestrel viene spesso messo dietro un reverse proxy?

> **Criterio di uscita:** sai leggere gli header di una richiesta reale e spiegare ogni riga, e sai dire a memoria quale status code restituire in otto situazioni diverse.

Sezione 4 — ASP.NET Core

30 ore · Modulo B · `NUCLEO`

OBIETTIVO

Il cuore tecnico di ogni colloquio backend .NET. La domanda sui service lifetime della dependency injection è praticamente garantita.

CONCETTI TEORICI DA PADRONEGGIARE

- **Hosting e avvio:** generic host, `Program.cs` moderno, ciclo di vita dell'applicazione, hosted service e background service.
- **Pipeline delle richieste:** middleware, ordine di registrazione e perché conta, cosa fa `next()`, terminal middleware, middleware custom.
- **Dependency Injection:** il container integrato; i tre lifetime (`Singleton`, `Scoped`, `Transient`) e le conseguenze pratiche di ognuno; captive dependency (Scoped dentro Singleton); registrazione di più implementazioni; factory delegate.
- **Configurazione:** gerarchia delle sorgenti (`appsettings.json`, ambiente, variabili d'ambiente, user secrets, Key Vault); Options pattern (`IOptions`, `IOptionsSnapshot`, `IOptionsMonitor`) e differenze; validazione della configurazione all'avvio.
- **API REST:** principi REST, design delle risorse e degli URL, verbi HTTP e loro semantica, idempotenza, status code corretti; controller contro Minimal API; model binding; validazione (DataAnnotations e FluentValidation); DTO e perché non esporre le entità; mapping; paginazione, filtri, ordinamento; versioning delle API; CORS; content negotiation.
- **Autenticazione e autorizzazione:** differenza tra le due; schema di autenticazione; **JWT** (struttura, firma, claim, scadenza, refresh token); cookie authentication; policy-based e role-based authorization; cenni a OAuth 2.0 e OpenID Connect.
- **Gestione errori:** middleware di eccezione globale, ProblemDetails (RFC 7807), cosa esporre e cosa no.
- **Logging e diagnostica:** `ILogger`, livelli, logging strutturato, correlation id, cenni a Serilog.
- **Performance e caching:** caching in memoria (`IMemoryCache`) contro caching distribuito (`IDistributedCache`, Redis); strategie di invalidazione e cache stampede; response caching; compressione; **health check**; rate limiting. Il caching è tra le competenze più citate e meno padroneggiate.
- **Librerie di uso corrente** che ricorrono negli annunci e nei progetti reali: FluentValidation (validazione), Serilog (logging strutturato), AutoMapper o mapping manuale, MediatR (CQRS leggero). Conoscerne l'esistenza e il problema che risolvono, non necessariamente padroneggiarle tutte.
- **Documentazione:** Swagger/OpenAPI.

ESERCIZI

1. **Middleware custom.** Scrivi una middleware che logga metodo, path, durata e status code di ogni richiesta, con un correlation id propagato. *Obiettivo:* capire la pipeline costruendoci dentro, ed è anche codice utile davvero.
2. **Esperimento sui lifetime.** Registra lo stesso servizio come Singleton, Scoped e Transient, iniettalo tre volte nella stessa richiesta e stampa gli hash code. Poi prova a iniettare uno Scoped dentro un Singleton e osserva l'errore. *Obiettivo:* rendere concreta la domanda da colloquio più frequente in assoluto.
3. **API completa.** Endpoint CRUD per le macchine con validazione, DTO, paginazione, filtri e status code corretti. *Obiettivo:* il pane quotidiano del mestiere.
4. **Autenticazione JWT da zero.** Endpoint di login, generazione del token, validazione, endpoint protetti, ruoli differenziati (operatore contro amministratore). Senza librerie che facciano tutto. *Obiettivo:* l'autenticazione è il tema su cui più spesso i junior hanno una comprensione superficiale.
5. **Gestione errori globale.** Middleware che intercetta le eccezioni e restituisce ProblemDetails, distinguendo errori di validazione, non trovato, non autorizzato e errore interno. *Obiettivo:* nessuna eccezione deve mai raggiungere il client così com'è.
6. **Background service.** Un servizio in background che ogni N secondi aggrega le letture e calcola gli allarmi. *Obiettivo:* concetto richiesto spesso e raramente conosciuto.

INCREMENTO SUL PROGETTO

MachineHub diventa un'API REST completa: registrazione macchine, ingestione letture, interrogazione produzione e allarmi, autenticazione JWT con ruoli, validazione, logging strutturato, gestione errori centralizzata, Swagger.

DOMANDE TIPICHE DA COLLOQUIO

- Spiegami la differenza tra Singleton, Scoped e Transient. Cosa succede se inietti uno Scoped in un Singleton?
- Descrivi il percorso di una richiesta HTTP dall'arrivo alla risposta.
- Perché l'ordine delle middleware conta? Fammi un esempio in cui invertirle rompe tutto.
- Come funziona un JWT? Cosa contiene, chi lo firma, come si valida, e cosa succede alla scadenza?
- Differenza tra autenticazione e autorizzazione.
- Che status code restituisci per: creazione riuscita, risorsa inesistente, validazione fallita, utente non autenticato, utente autenticato ma senza permessi?
- Perché usare DTO invece di esporre direttamente le entità?
- Cos'è l'idempotenza e quali verbi HTTP dovrebbero esserlo?

> **Criterio di uscita:** l'API gira, è autenticata e documentata, e sai raccontare a voce l'intero ciclo di vita di una richiesta senza guardare il codice.

Sezione 5 — Entity Framework Core

20 ore · Modulo B · **NUCLEO**

OBIETTIVO

Usare un ORM senza subirlo. Il problema N+1 è la domanda trabocchetto più comune dell'argomento, e la SQL della sezione 3 è ciò che ti permette di capire cosa EF sta facendo davvero.

CONCETTI TEORICI DA PADRONEGGIARE

- `DbContext`**:** ciclo di vita, perché registrato come Scoped, unit of work implicito, `SaveChanges` e transazioni implicite.
- **Change tracking:** stati delle entità (Added, Modified, Deleted, Unchanged, Detached), `AsNoTracking` e quando usarlo, costo del tracking su grandi risultati.
- **Mapping:** convenzioni, data annotation contro Fluent API, configurazione delle relazioni (uno-a-molti, molti-a-molti, uno-a-uno), chiavi composite, owned entity, conversioni di valore, shadow property.
- **Migrations:** creazione, applicazione, rollback, migration a mano, gestione in team e in pipeline, seeding.
- **Caricamento dei dati:** lazy loading, eager loading (`Include` , `ThenInclude`), explicit loading; **il problema N+1**: come nasce, come riconoscerlo nei log, come risolverlo; split query contro single query e cartesian explosion.
- **Query:** traduzione LINQ→SQL e cosa non è traducibile; valutazione lato client e perché è pericolosa; query compilate; proiezioni per ridurre i dati trasferiti.
- **Scrittura:** insert massivi e loro costo, `ExecuteUpdate` e `ExecuteDelete` , concorrenza ottimistica con token di concorrenza.
- **Quando non usare EF:** query complesse, report, operazioni bulk; alternative (SQL grezzo, Dapper).
- **Osservabilità:** logging delle query generate.

ESERCIZI

1. **N+1 provocato.** Scrivi codice che genera un N+1, mostralo nei log SQL contando le query, poi correggilo con `Include` e verifica il conteggio. *Obiettivo:* è il problema di performance più comune in ogni applicazione EF esistente.
2. **Confronto EF contro SQL.** Scrivi la stessa query di report in EF e in SQL grezzo, confronta il SQL generato e i tempi. Documenta quale sceglieresti e perché. *Obiettivo:* saper motivare la scelta è ciò che distingue chi usa EF consapevolmente.
3. **Migration manuale.** Crea una situazione in cui la migration generata automaticamente non basta (per esempio un cambio di tipo con conversione di dati) e scrivila a mano. *Obiettivo:* succede sempre nei progetti reali.
4. **Concorrenza ottimistica.** Implementa un token di concorrenza e provoca deliberatamente un conflitto tra due aggiornamenti. *Obiettivo:* tema tipico dei sistemi multiutente.
5. **Tracking a confronto.** Misura una query di sola lettura con e senza `AsNoTracking` su un risultato ampio. *Obiettivo:* ottimizzazione a costo zero che quasi nessuno applica.

INCREMENTO SUL PROGETTO

Sostituzione della persistenza in memoria con EF Core sul database della sezione 3. Migrations funzionanti, repository reali, almeno una query di reporting complessa realizzata in SQL grezzo con motivazione documentata.

DOMANDE TIPICHE DA COLLOQUIO

• Cos'è il problema N+1? Come lo riconosci e come lo risolvi?
• Cos'è il change tracking e quando lo disattiveresti?
• Differenza tra lazy, eager ed explicit loading.
• Perché il `DbContext` è Scoped e cosa succederebbe se fosse Singleton?
• Quando abbandoneresti EF per SQL grezzo o Dapper?
• Cos'è la concorrenza ottimistica e come la implementi?
• Come gestisci le migrations in un team di cinque persone?

> **Criterio di uscita:** sai mostrare i log SQL di un N+1 provocato e risolto da te, e sai motivare un caso in cui hai scelto SQL grezzo.

Sezione 5bis — Sicurezza applicativa

8 ore · Modulo B · **NUCLEO**

OBIETTIVO

Questa **non è una specializzazione**: è la baseline che ci si aspetta da qualunque backendista. Un candidato che non sa dove va conservato un token, o che hasha le password con SHA-256, viene scartato indipendentemente da quanto sia bravo sul resto. La cloud security e il DevSecOps restano roadmap separate; questo è il minimo professionale.

CONCETTI TEORICI DA PADRONEGGIARE

- **OWASP Top 10 in chiave ASP.NET Core:** per ciascuna categoria, come si manifesta concretamente in un'applicazione .NET e quale contromisura offre il framework.
- **Password:** perché non si cifrano ma si hashano; perché SHA-256 da solo non basta; funzioni lente e salate — **BCrypt**, **Argon2**, PBKDF2 (usato da ASP.NET Core Identity); work factor; salt e pepper.
- **Token lato client:** dove sta un JWT e i rischi di ciascuna scelta (localStorage e XSS, cookie `HttpOnly` e CSRF); durata breve dell'access token; **refresh token con rotazione** e rilevamento del riuso; revoca e blacklist; cosa non mettere mai dentro un JWT.
- **Controllo degli accessi: IDOR** (broken object level authorization) — la vulnerabilità più comune e più ignorata nelle API; autorizzazione a livello di risorsa e non solo di endpoint; privilege escalation orizzontale e verticale.
- **Injection:** SQL injection anche con EF quando si usa `FromSqlRaw` o l'interpolazione non parametrizzata; command injection; path traversal.
- **XSS e CSRF:** come nascono, encoding dell'output, Content Security Policy, antiforgery token in ASP.NET Core, e perché le API con token sono meno esposte al CSRF ma non immuni.
- **Mass assignment / over-posting:** perché legare direttamente le entità nel model binding è pericoloso e perché i DTO sono anche una misura di sicurezza.
- **Abuso e disponibilità:** rate limiting, lockout dopo tentativi falliti, limiti di dimensione delle richieste, protezione contro l'enumerazione degli utenti nei messaggi d'errore.
- **Segreti:** mai nel codice né nel repository; user secrets in sviluppo, variabili d'ambiente e vault in produzione; cosa fare se un segreto finisce in un commit; scansione automatica dei segreti.
- **Dipendenze:** vulnerabilità note nei pacchetti NuGet, `dotnet list package --vulnerable`, Dependabot, cenni a SBOM e supply chain.
- **Logging sicuro:** cosa non deve mai finire nei log (password, token, dati personali, numeri di carta); log injection; cosa esporre in un messaggio d'errore al client contro cosa tenere nei log interni.
- **Header di sicurezza:** HSTS, `X-Content-Type-Options`, `X-Frame-Options`, CSP.

ESERCIZI

1. **Attacca la tua stessa API.** Prova a leggere i dati di un'altra macchina cambiando l'id nell'URL con un utente che non ne ha diritto. *Obiettivo:* quasi certamente funzionerà. L'IDOR è il buco più comune nelle API scritte da sviluppatori competenti, e trovarlo nel proprio codice è la lezione che resta.
2. **Autorizzazione a livello di risorsa.** Correggi il punto sopra verificando la proprietà della risorsa, non solo il ruolo. *Obiettivo:* capire la differenza tra autenticato, autorizzato all'endpoint e autorizzato *a quella* risorsa.
3. **Hashing.** Sostituisci un ipotetico hashing ingenuo con BCrypt o Argon2, misura il tempo di verifica e taralo. *Obiettivo:* capire perché la lentezza è una caratteristica desiderata, non un difetto.
4. **Refresh token con rotazione.** Implementa access token a vita breve e refresh token ruotanti, con rilevamento del riuso di un token già speso. *Obiettivo:* è lo schema che ti verrà chiesto di descrivere e che quasi nessun junior sa disegnare.
5. **Injection con EF.** Scrivi una `FromSqlRaw` vulnerabile, sfruttala, poi correggila con parametri. *Obiettivo:* smontare l'illusione che usare un ORM renda automaticamente immuni.
6. **Mass assignment.** Prova a promuovere il tuo utente a amministratore inviando un campo in più nel body. *Obiettivo:* vedere perché i DTO non sono solo una questione di pulizia.
7. **Dipendenze.** Esegui la scansione delle vulnerabilità sui pacchetti del progetto e risolvi quelle trovate. *Obiettivo:* abitudine da prendere una volta e mantenere per sempre.

INCREMENTO SUL PROGETTO

MachineHub diventa difendibile: autorizzazione a livello di risorsa su ogni endpoint, password hashate correttamente, refresh token con rotazione, rate limiting sul login, header di sicurezza, nessun segreto nel repository, logging ripulito dai dati sensibili. Apri il file `docs/security.md` e inizia a documentare ogni misura adottata e la minaccia che neutralizza.

DOMANDE TIPICHE DA COLLOQUIO

• Come conservi le password degli utenti? Perché non SHA-256?
• Dove salvi un JWT lato client e quali rischi comporta la tua scelta?
• Cos'è un refresh token e perché andrebbe ruotato?
• Cos'è un IDOR e come lo previeni?
• Usando Entity Framework sono al sicuro dalla SQL injection?
• Differenza tra XSS e CSRF; le API con token sono esposte al CSRF?
• Cos'è il mass assignment e come lo eviti?

- Hai committato per sbaglio una chiave API: quali sono i passi?
- Cosa non deve mai finire in un log?

> **Criterio di uscita:** hai trovato e chiuso almeno una vulnerabilità reale nel tuo codice, `docs/security.md` esiste e documenta le contromisure, e sai raccontare in due minuti come funziona il tuo schema di autenticazione.

Sezione 5ter — Real-time con SignalR

4 ore · Modulo B · `COMPRIMIBILE`

> **Se la salti, cosa perdi.** Nulla sul piano delle competenze richieste: il real-time non è un requisito comune per un backend .NET. Perdi però l'unica schermata di MachineHub che un selezionatore può *guardare* in dieci secondi, e con essa lo screenshot in cima al README. È la prima sezione da tagliare se il tempo stringe, e quella da recuperare per prima quando ne hai.

OBIETTIVO

Sezione breve con un ritorno sproporzionato sul portfolio: trasforma MachineHub da API interrogabile a **sistema che si vede muovere**. Una dashboard con macchine e allarmi che si aggiornano dal vivo è qualcosa che un selezionatore può guardare in dieci secondi, ed è il caso d'uso naturale del dominio industriale.

CONCETTI TEORICI DA PADRONEGGIARE

- **Le tre strade verso il real-time:** polling e long polling, **Server-Sent Events**, **WebSocket** — costi, limiti e quando ciascuna è la scelta giusta. SignalR come astrazione che negozia il trasporto migliore disponibile.
- **Hub:** metodi server chiamabili dal client e viceversa; ciclo di vita della connessione; `OnConnectedAsync` e `OnDisconnectedAsync`; connection id.
- **Destinatari:** broadcast a tutti, a un singolo utente, a **gruppi** (per esempio un gruppo per reparto o per macchina).
- **Autenticazione e autorizzazione sulla connessione:** come viaggia il token su WebSocket (query string e sue implicazioni), attributo `[Authorize]` sugli hub, mappatura utente→connessioni.
- **Affidabilità:** riconnessione automatica, gestione dello stato alla riconnessione, messaggi persi e perché il real-time non sostituisce una fonte di verità interrogabile.
- **Scalabilità:** perché più istanze rompono il broadcast, cos'è un **backplane** (Redis, Azure SignalR Service), sticky session.

ESERCIZI

1. **Hub base.** Un hub che notifica in tempo reale ogni nuova lettura ricevuta, con una pagina HTML minimale che la mostra. *Obiettivo:* vedere il meccanismo funzionare end-to-end.
2. **Gruppi.** Un client si iscrive solo alle macchine di un reparto e riceve solo quelle. *Obiettivo:* i gruppi sono ciò che rende SignalR utile invece che rumoroso.
3. **Connessione autenticata.** Proteggi l'hub e verifica che una connessione senza token venga rifiutata. *Obiettivo:* il real-time è una porta d'ingresso che viene dimenticata spesso.
4. **Resilienza.** Stacca la rete, riattaccala, e assicurati che il client si riconnetta e si riallinei allo stato corrente. *Obiettivo:* capire che la riconnessione da sola non recupera i messaggi persi.

INCREMENTO SUL PROGETTO

Dashboard live di MachineHub: stato delle macchine, contatori di produzione e allarmi che si aggiornano in tempo reale, con sottoscrizione per reparto e connessione autenticata. È la schermata da mettere come screenshot in cima al README.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra polling, SSE e WebSocket. Quando sceglieresti l'uno o l'altro?

• Come autentichi una connessione WebSocket?

• Cosa succede a SignalR quando scali su tre istanze?

• Un client si disconnette per trenta secondi: cosa succede ai messaggi di quel periodo?

> **Criterio di uscita:** la dashboard si aggiorna dal vivo, la connessione è autenticata, e sai spiegare cosa servirebbe per farla funzionare su più istanze.

> **Traguardo raggiunto — soglia di candidabilità (157 ore, circa 4 mesi).** Con il Modulo B completo sei candidabile a posizioni Junior/Mid .NET Developer, Backend Developer C#, ERP Developer su piattaforme .NET, Application Developer, e ai programmi Trainee e Graduate. Da qui in avanti si studia e ci si candida in parallelo.
>
> **Prima di considerare chiuso il modulo, i tre artefatti obbligatori:** README presentabile, deploy minimo pubblico raggiungibile via URL, Swagger navigabile su quell'ambiente. Senza il link, le candidature dei mesi successivi partono senza nulla da mostrare.

MODULO C — Professionalità

Sezioni 6 → 7bis · 31 ore, di cui 23 di nucleo · Il miglior rapporto tra ore investite e credibilità guadagnata.

Sezione 6 — Testing

15 ore · Modulo C · `NUCLEO`

OBIETTIVO

La maggior parte degli sviluppatori junior non scrive test. Saperlo fare è un salto di credibilità sproporzionato rispetto alle ore che costa. E risolve direttamente il problema della fiducia in sé: smetti di sperare che funzioni e lo verifichi.

CONCETTI TEORICI DA PADRONEGGIARE

- **Tipologie di test:** unit, integration, end-to-end; la piramide dei test e le sue critiche; cosa vale la pena testare e cosa no.
- **xUnit:** struttura Arrange-Act-Assert; `[Fact]` contro `[Theory]` con `[InlineData]` e `[MemberData]`; fixture e ciclo di vita; asserzioni; test asincroni.
- **Test double:** dummy, stub, mock, spy, fake — le differenze; Moq o NSubstitute; cosa mockare e cosa no; perché mockare tutto è un anti-pattern.
- **Testabilità:** cosa rende una classe difficile da testare (dipendenze statiche, `DateTime.Now`, `new` interno, side effect nascosti) e come rimediare.
- **Test di integrazione:** `WebApplicationFactory`, database in memoria contro container reale (Testcontainers), gestione dello stato tra test.
- **Metriche:** code coverage come indicatore e non come obiettivo; test fragili; test lenti.
- **Cenni a TDD:** il ciclo red-green-refactor, e quando applicarlo ha senso.

ESERCIZI

1. **Unit test sul dominio.** Copertura completa della logica di calcolo dell'efficienza e di generazione allarmi, inclusi i casi limite. *Obiettivo:* la logica di dominio è ciò che vale davvero la pena testare.
2. **Refactoring per testabilità.** Prendi una classe che usa `DateTime.Now` internamente e rendila testabile. *Obiettivo:* esempio canonico, e capita in ogni codebase.
3. **Mocking mirato.** Testa un servizio applicativo mockando solo il repository, non tutto. *Obiettivo:* imparare la misura, che è la parte difficile.
4. **Test di integrazione.** Test che avvia l'API in memoria, chiama un endpoint autenticato e verifica la risposta e lo stato del database. *Obiettivo:* è ciò che dà vera fiducia prima di un rilascio.
5. **Prova del nove.** Rompi deliberatamente una regola di business e verifica che i test la intercettino. *Obiettivo:* un test che non fallisce mai non sta testando nulla.

INCREMENTO SUL PROGETTO

Suite di test su MachineHub, tutti verdi, con badge di copertura nel README.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra unit test e integration test.
• Cosa mockeresti e cosa no?
• Come testi codice che dipende dall'ora corrente o da un servizio esterno?
• Il 100% di code coverage significa che il codice è corretto?
• Cosa rende un test fragile?

> **Criterio di uscita:** puoi modificare la logica di business e sapere entro trenta secondi, senza avviare l'applicazione, se hai rotto qualcosa.

Sezione 6bis — Diagnostica e performance applicativa

6 ore · Modulo C · `CONSIGLIATO`

OBIETTIVO

Nota onesta sulle ore: sei ore su questo argomento sono un assaggio, non una competenza mid-level. Il ritorno vero non è padroneggiare la diagnostica — per quella servono anni sul campo — ma acquisire il metodo e portarsi a casa un racconto da colloquio con numeri veri. Con quell'aspettativa, sono sei ore ben spese.

È il confine più netto tra un profilo entry e uno mid. Negli annunci compare come "troubleshooting in produzione" o "performance optimization", e distingue chi sa scrivere codice da chi sa capire perché il codice si comporta male quando è sotto carico e non sul suo portatile.

CONCETTI TEORICI DA PADRONEGGIARE

- **Strumenti diagnostici .NET:** `dotnet-counters` (metriche live), `dotnet-trace` (tracce di esecuzione), `dotnet-dump` (analisi di un dump di memoria), `dotnet-gcdump`. Come si usano su un processo in esecuzione, anche dentro un container.
- **Misurazione rigorosa: BenchmarkDotNet**; perché misurare con un cronometro nel codice porta a conclusioni sbagliate (JIT, warm-up, garbage collection).
- **Memoria:** generazioni del garbage collector, Large Object Heap, **memory leak** in un linguaggio gestito (eventi non disconnessi, cache senza scadenza, statiche che trattengono riferimenti), come si riconosce un leak da un grafico di memoria.
- **Concorrenza sotto carico: thread pool starvation** (causa tipica: codice sincrono su chiamate asincrone), **socket exhaustion** e perché `HttpClient` va usato tramite `IHttpClientFactory`, contesa sui lock.
- **Metriche che contano:** latenza **p50, p95, p99** e perché la media mente; throughput; tasso di errore; saturazione. I quattro segnali d'oro.
- **Load testing:** concetti di carico, stress, soak; strumenti (k6, NBomber, JMeter); come si progetta un test che dice qualcosa di vero.
- **Osservabilità:** le tre gambe (log, metriche, tracce); **OpenTelemetry** come standard indipendente dal fornitore; tracce distribuite e correlation id; strumentazione automatica contro manuale.
- **Ottimizzazione ragionata:** misurare prima, ottimizzare il collo di bottiglia reale, rimisurare. Perché l'ottimizzazione a intuito è quasi sempre sprecata.

ESERCIZI

1. **Benchmark onesto.** Confronta due implementazioni della stessa funzione con BenchmarkDotNet, poi rifai la stessa misura con un cronometro artigianale e osserva quanto divergono i risultati. *Obiettivo:* capire perché serve uno strumento serio.
2. **Leak provocato.** Crea deliberatamente un memory leak (per esempio una cache statica che cresce senza limite), osservalo con `dotnet-counters`, catturane un dump e trovane la causa. *Obiettivo:* l'unico modo di imparare a leggere un dump è averne uno di cui conosci già la risposta.
3. **Starvation.** Provoca thread pool starvation chiamando codice asincrono in modo sincrono sotto carico, osserva il crollo, e correggilo. *Obiettivo:* collega direttamente l'asincronia della Sezione 1 a un fallimento reale in produzione.
4. **Test di carico.** Metti sotto carico un endpoint di MachineHub, registra p50, p95 e p99, trova il collo di bottiglia, ottimizzalo e rimisura. Documenta i numeri prima e dopo. *Obiettivo:* questo esercizio produce uno dei tre racconti da colloquio della Sezione 12.
5. **Tracciamento.** Attiva OpenTelemetry sull'applicazione e segui una richiesta attraverso API, database e coda. *Obiettivo:* vedere il sistema come lo vedrai in produzione.

INCREMENTO SUL PROGETTO

MachineHub strumentato con OpenTelemetry, con un file `docs/performance.md` che riporta i risultati del test di carico, il collo di bottiglia individuato, l'intervento fatto e i numeri prima e dopo.

DOMANDE TIPICHE DA COLLOQUIO

• L'applicazione in produzione è lenta. Come procedi, concretamente?
• Perché si guardano p95 e p99 e non la media?
• Esiste un memory leak in .NET, che ha il garbage collector? Come si genera?
• Cos'è la thread pool starvation e quale errore la causa tipicamente?
• Perché `HttpClient` non va istanziato con `new` a ogni chiamata?
• Cosa sono log, metriche e tracce e a cosa serve ciascuno?

> **Criterio di uscita:** hai un'ottimizzazione documentata con numeri prima e dopo, e sai raccontare in due minuti come diagnosticheresti un rallentamento in produzione senza avere accesso al debugger.

Sezione 7 — Git avanzato e collaborazione

8 ore · Modulo C · `NUCLEO`

OBIETTIVO

Poche ore, rendimento alto. Compare in ogni annuncio e in ogni team ti si chiede di lavorare così.

CONCETTI TEORICI DA PADRONEGGIARE

- **Modello interno di Git:** commit, albero, blob, riferimenti, HEAD. Capire il modello rende tutto il resto ovvio.
- **Strategie di branching:** Git Flow, GitHub Flow, trunk-based development; pro e contro.
- `merge` **contro** `rebase`**:** cosa fanno alla storia, fast-forward, squash, quando usare quale, la regola d'oro del rebase su branch condivisi.
- **Comandi di recupero e manutenzione:** `stash`, `cherry-pick`, `reset` (soft, mixed, hard), `revert`, `reflog`, `bisect`, rebase interattivo.
- **Collaborazione:** pull request, code review (come si fa e come si riceve), branch protection, conventional commits.

ESERCIZI

1. **Storia pulita.** Prendi una serie di commit disordinati e riscrivila con rebase interattivo. *Obiettivo:* la storia del repository fa parte del portfolio.
2. **Recupero.** Fai un `reset --hard` che distrugge del lavoro e recuperalo con `reflog`. *Obiettivo:* togliersi la paura di Git.
3. **Caccia al bug.** Introduci un bug qualche commit indietro e trovalo con `bisect`. *Obiettivo:* strumento poco noto e molto utile.
4. **Conflitto reale.** Crea due branch che modificano le stesse righe e risolvi il conflitto capendo cosa stai facendo.

INCREMENTO SUL PROGETTO

Da qui in avanti si lavora su MachineHub solo tramite feature branch e pull request verso `main`, anche lavorando da soli, con branch protection attiva.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra merge e rebase. Quando useresti l'uno o l'altro?
• Cosa fa `reset --soft` contro `--hard`?
• Hai committato una password per sbaglio: cosa fai?
• Come organizzeresti i branch in un team di cinque persone?

> **Criterio di uscita:** la storia di MachineHub è leggibile e nessun commit si chiama "fix" o "update".

Sezione 7bis — Agile e Scrum

2 ore · Modulo C · **CONSIGLIATO**

OBIETTIVO

Due ore, e sono tra le più redditizie del documento. La metodologia agile compare in quasi ogni annuncio enterprise, e il colloquio quasi sempre include una domanda su come si lavora in team. Non saper rispondere è un filtro banale ma reale, e si elimina in una serata.

CONCETTI TEORICI DA PADRONEGGIARE

- **Il manifesto agile** in sintesi: i quattro valori, e cosa significano in pratica al di là dello slogan.
- **Ruoli Scrum:** Product Owner, Scrum Master, Development Team — responsabilità di ciascuno e cosa *non* è compito loro.
- **Artefatti:** product backlog, sprint backlog, incremento; definition of ready e **definition of done**.
- **Eventi:** sprint planning, daily stand-up, sprint review, retrospettiva, refinement — durata, scopo e l'errore tipico di ciascuno (il daily che diventa un rapporto al capo, la retrospettiva senza azioni).
- **Stima:** user story e formato "come [ruolo] voglio [azione] per [beneficio]"; criteri di accettazione; **story point** e perché non sono ore; planning poker; velocity e come viene abusata.
- **Kanban:** flusso continuo, limiti al work in progress, lead time e cycle time; quando Kanban è più adatto di Scrum.
- **Strumenti:** Azure Boards, GitHub Projects, Jira — board, colonne, collegamento tra commit/PR e work item.

ESERCIZI

1. **Backlog reale.** Costruisci il backlog di MachineHub su GitHub Projects o Azure Boards: le funzionalità rimanenti scritte come user story con criteri di accettazione. *Obiettivo:* scrivere una user story decente è più difficile di quanto sembri, e si impara solo facendolo.
2. **Sprint su te stesso.** Pianifica le prossime due settimane di roadmap come uno sprint: obiettivo, backlog stimato in story point, definition of done. Alla fine fai una retrospettiva scritta. *Obiettivo:* il metodo funziona anche su un team di uno, e ti dà qualcosa di vero da raccontare.
3. **Collegamento.** Collega commit e pull request ai work item corrispondenti. *Obiettivo:* è la pratica che ti chiederanno dal primo giorno in azienda.

INCREMENTO SUL PROGETTO

Board pubblica di MachineHub con backlog, sprint in corso e work item collegati alle PR. È anche un segnale di serieta visibile a chi guarda il repository.

DOMANDE TIPICHE DA COLLOQUIO

- Hai lavorato in Scrum? Descrivimi come si svolgeva uno sprint.
- Che differenza c'è tra Product Owner e Scrum Master?
- Cos'è la definition of done e a cosa serve?
- Perché si stima in story point invece che in ore?
- Quando useresti Kanban invece di Scrum?
- A cosa serve la retrospettiva e cosa la rende inutile?

> **Criterio di uscita:** sai descrivere uno sprint dall'inizio alla fine usando la terminologia corretta, e hai una board reale da mostrare.

MODULO D — Cloud e delivery

Sezioni 8 → 11bis · 98 ore, di cui 62 di nucleo · La trasversalità: qui si apre la strada a DevOps e DevSecOps.

> **Nota sulla profondità di questo modulo.** Le sezioni che seguono coprono ciò che serve a un *backend developer* per essere completo e assumibile: containerizzare, automatizzare, deployare, integrare. Non coprono la specializzazione DevOps (Kubernetes, Terraform, infrastructure as code avanzata, observability) né la cloud security. Quelle meritano roadmap dedicate, da affrontare solo dopo aver completato questa.

Sezione 8 — Docker e container

15 ore · Modulo D · **NUCLEO**

CONCETTI TEORICI DA PADRONEGGIARE

- Cos'è un container e in cosa differisce da una macchina virtuale; namespace e cgroup in sintesi.
- Immagini, layer, cache di build, e perché l'ordine delle istruzioni nel Dockerfile determina i tempi di build.
- Dockerfile: istruzioni principali, **multi-stage build**, scelta dell'immagine base, `.dockerignore`.
- Runtime: volumi (bind mount contro named volume), network, variabili d'ambiente, port mapping, gestione dei log.
- `docker-compose`: servizi, dipendenze, health check, orchestrazione locale.
- Registry: push, pull, tagging e versionamento delle immagini.
- Sicurezza di base: non eseguire come root, immagini minimali, scansione delle vulnerabilità, gestione dei segreti (mai nel Dockerfile).
- Cenni all'orchestrazione (Kubernetes) solo per sapere cosa risolve.
- **.NET Aspire:** l'approccio proposto da Microsoft per orchestrare localmente applicazioni .NET cloud-native. Basta conoscerlo e sapere che problema risolve; approfondirlo è opzionale.

ESERCIZI

1. **Dockerfile a mano.** Scrivi il Dockerfile di MachineHub senza generarlo dall'IDE, prima single-stage poi multi-stage, e confronta la dimensione dell'immagine. *Obiettivo:* vedere il vantaggio invece di applicarlo per fede.
2. **Stack completo.** `docker-compose.yml` che avvia API, SQL Server e broker di messaggi, con health check e ordine di avvio corretto. *Obiettivo:* chiunque cloni il repository deve poter avviare tutto con un comando.
3. **Persistenza.** Configura un volume perché i dati del database sopravvivano al riavvio del container, poi verificalo. *Obiettivo:* errore classico di chi inizia.
4. **Hardening minimo.** Fai girare il container con un utente non-root e verifica che funzioni.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra container e macchina virtuale.
• Cos'è una build multi-stage e perché conviene?
• Perché l'ordine delle istruzioni nel Dockerfile influenza i tempi di build?
• Come gestisci i segreti in un container?
• Come fai persistere i dati di un database containerizzato?

> **Criterio di uscita:** su una macchina pulita, `git clone` seguito da `docker compose up` produce l'applicazione funzionante.

Sezione 9 — CI/CD

12 ore · Modulo D · NUCLEO

CONCETTI TEORICI DA PADRONEGGIARE

- Continuous integration, continuous delivery e continuous deployment: le tre cose sono diverse.
- **GitHub Actions** (o Azure DevOps Pipelines): sintassi YAML, trigger, job, step, runner, matrici, artefatti, cache, dipendenze tra job.
- Pipeline di build: restore, build, test, analisi statica, pubblicazione artefatto.
- Pipeline di deploy: ambienti, approvazioni manuali, deploy blue-green e canary in sintesi, rollback.
- Gestione dei segreti nella pipeline; principio del privilegio minimo per le credenziali di deploy.
- Quality gate: branch protection, test obbligatori, soglie di copertura, revisione obbligatoria.
- Build e push dell'immagine Docker verso un registry; versionamento semantico.

ESERCIZI

1. **Pipeline di CI da zero.** Scritta a mano in YAML: a ogni push compila, esegue i test, pubblica il risultato. *Obiettivo:* scriverla, non copiarla, è l'unico modo per saperla poi modificare.
2. **Quality gate.** Configura branch protection su `main`, poi apri una PR con un test volutamente rotto e verifica che il merge sia impossibile. *Obiettivo:* vedere il meccanismo funzionare.
3. **Build e push immagine.** Estendi la pipeline perché costruisca l'immagine Docker e la pubblichi su un registry con tag versionato.
4. **Diagnosi.** Rompi deliberatamente la pipeline in tre modi diversi e impara a leggere i log per capire quale.

DOMANDE TIPICHE DA COLLOQUIO

• Differenza tra continuous delivery e continuous deployment.
• Cosa metteresti in una pipeline di CI per un progetto .NET?
• Come gestisci i segreti in una pipeline?
• Come faresti un rollback di un deploy andato male?

> **Criterio di uscita:** ogni PR su MachineHub viene verificata automaticamente, e il badge di stato è nel README.

Sezione 10 — Azure

35 ore · Modulo D · NUCLEO

OBIETTIVO

La competenza con il ritorno più alto della roadmap. È anche la sezione più lunga, e volutamente: l'elenco di argomenti che segue non sta in venti ore, e sottostimarla significherebbe attraversarla senza mai mettere le mani su metà dei servizi. Nell'ecosistema Microsoft la fluidità su Azure è ormai requisito di base per i ruoli backend, ed è la porta d'ingresso a DevOps e cloud security.

CONCETTI TEORICI DA PADRONEGGIARE

- **Fondamentali:** modelli di servizio (IaaS, PaaS, SaaS, serverless); regioni e zone di disponibilità; resource group, sottoscrizioni, tenant; Azure Resource Manager; portale, CLI e Bicep/ARM in sintesi.
- **Compute:** App Service (piani, slot di staging, scaling verticale e orizzontale, autoscale); Azure Functions (trigger, binding, piani di hosting, cold start); Container Apps in sintesi.
- **Dati:** Azure SQL Database (tier, DTU contro vCore, backup, geo-replica); Blob Storage (tier di accesso, SAS token); cenni a Cosmos DB e Redis Cache.
- **Identità e segreti:** Microsoft Entra ID; service principal; **managed identity**; **Key Vault**; RBAC e principio del privilegio minimo. L'obiettivo pratico: nessuna credenziale nel codice né nei file di configurazione.
- **Integrazione:** Service Bus, Event Grid, Event Hubs — a cosa serve ciascuno; API Management in sintesi.
- **Osservabilità:** Application Insights (tracce distribuite, dipendenze, live metrics), Log Analytics e query KQL di base, alert.
- **Rete:** VNet, subnet, network security group, private endpoint, regole firewall del database.
- **Governance e costi:** tag, budget, alert di spesa, come si legge una stima.

ESERCIZI

1. **Deploy manuale, poi automatico.** Prima pubblica MachineHub su App Service a mano dal portale, poi automatizzalo dalla pipeline. *Obiettivo:* fare la cosa a mano una volta insegna cosa sta automatizzando la pipeline.
2. **Database in cloud.** Migra il database su Azure SQL, configura il firewall, verifica le migrations in ambiente cloud.
3. **Zero segreti.** Sposta ogni stringa di connessione e chiave in Key Vault, accedendovi con managed identity. Verifica che il repository non contenga nemmeno una credenziale. *Obiettivo:* è la pratica che più distingue un profilo maturo, ed è il primo mattone della cloud security.
4. **Diagnosi in produzione.** Introduci deliberatamente un endpoint lento, trovalo in Application Insights e mostra la traccia. *Obiettivo:* saper diagnosticare in cloud è ciò che ti rende affidabile su un sistema reale.

5. **Serverless.** Una Azure Function con trigger temporale che genera il report giornaliero di produzione.

> **Attenzione ai costi.** Usa la sottoscrizione gratuita e i tier di sviluppo, imposta subito un budget con alert, e spegni o elimina le risorse quando non stai studiando. Una risorsa dimenticata accesa è l'errore più comune e più caro.

DOMANDE TIPICHE DA COLLOQUIO

- Differenza tra IaaS, PaaS e serverless, con un esempio Azure per ciascuno.
- Come gestisci i segreti di un'applicazione in Azure?
- Cos'è una managed identity e che problema risolve?
- Come faresti scalare un'API sotto carico?
- A cosa serve uno slot di staging?
- Come diagnostichi un'applicazione lenta in produzione?

CERTIFICAZIONI (OPZIONALI MA CONSIGLIATE)

AZ-900 Azure Fundamentals (~$99, spesso gratuita tramite i Microsoft Virtual Training Days) come primo timbro, subito dopo questa sezione. Più avanti **AZ-204 Azure Developer Associate** (~$165): è la certificazione Azure pensata per chi sviluppa ed è quella che pesa sul CV. Verificare sempre codici e prezzi sul sito Microsoft, perché cambiano.

> **Criterio di uscita:** MachineHub gira in cloud, raggiungibile via internet, il deploy è automatico dalla pipeline, e nel repository non esiste una sola credenziale.

Sezione 10bis — NoSQL, il minimo indispensabile

3 ore · Modulo D · `COMPRIMIBILE`

> **Se la salti, cosa perdi.** Tre ore non bastano a imparare NoSQL: quello che ottieni è consapevolezza, non conoscenza — abbastanza per non restare muto a una domanda, non per progettare su un documentale. Se la tagli, recupera almeno la voce NoSQL leggendo l'Appendice. L'unica parte con valore operativo immediato è Redis per il caching: quella tienila comunque, anche saltando il resto.

OBIETTIVO

Non diventare uno specialista di NoSQL, ma smettere di essere quello che non sa rispondere quando gliene chiedono. Compare regolarmente tra le competenze richieste accanto ad Azure e ai microservizi, e tre ore bastano a coprire il livello atteso.

CONCETTI TEORICI DA PADRONEGGIARE

- **Le famiglie:** documentale (MongoDB, Cosmos DB), chiave-valore (Redis, DynamoDB), a colonne (Cassandra), a grafo (Neo4j) — quale problema risolve ciascuna.
- **Modellazione a documenti:** denormalizzazione voluta, embedding contro referencing, progettare intorno alle query invece che intorno ai dati. È il ribaltamento mentale rispetto al relazionale.
- **Distribuzione:** teorema **CAP** e cosa si sceglie davvero nella pratica; consistenza forte contro eventuale; livelli di consistenza in Cosmos DB.
- **Partizionamento: partition key** — come si sceglie, cosa sono le hot partition, perché una scelta sbagliata è costosa e difficile da correggere dopo.
- **Costo: RU/s** in Cosmos DB, throughput provisionato contro serverless, throttling e `429`.
- **Quando NON usarlo:** transazioni tra entità diverse, query ad hoc imprevedibili, forti requisiti di integrità referenziale, dati fortemente relazionali. Il caso più comune resta il relazionale, e saperlo dire è un punto a favore, non contro.
- **Redis** come caso pratico immediato: caching distribuito, strutture dati offerte, scadenze, e il ruolo di backplane visto nella Sezione 5ter.

ESERCIZI

1. **Stesso dominio, due modelli.** Modella le letture di MachineHub come documenti e confronta il risultato con lo schema relazionale della Sezione 3. Elenca cosa diventa più facile e cosa più difficile. *Obiettivo:* il confronto diretto insegna più di qualsiasi definizione.
2. **Partition key.** Scegli una partition key per le letture, poi individua uno scenario che genera una hot partition. *Obiettivo:* è la domanda tecnica più probabile sull'argomento.
3. **Redis in pratica.** Metti in cache su Redis il report di produzione più costoso di MachineHub, con scadenza e invalidazione. *Obiettivo:* collega NoSQL, caching e performance in un unico intervento utile davvero.

INCREMENTO SUL PROGETTO

Caching distribuito su Redis per i report pesanti, con strategia di invalidazione documentata. Opzionale: una collezione documentale per lo storico grezzo delle letture, come esperimento comparativo.

DOMANDE TIPICHE DA COLLOQUIO

• Quando sceglieresti NoSQL e quando un relazionale?
• Cos'è una partition key e cosa succede se la scegli male?
• Consistenza forte contro eventuale: cosa comporta per l'applicazione?
• Come modelleresti una relazione uno-a-molti in un database documentale?
• A cosa usi Redis in un'applicazione .NET?

> **Criterio di uscita:** Redis è in uso su MachineHub, e sai argomentare in due minuti perché per questo progetto il relazionale resta la scelta principale.

Sezione 11 — Messaggistica asincrona

12 ore · Modulo D · `CONSIGLIATO`

OBIETTIVO

Compare come requisito negli annunci enterprise e nel remoto internazionale. È il concetto che separa chi sa fare un CRUD da chi sa progettare un sistema distribuito. Ha inoltre un valore doppio in ambito industriale, dove MQTT è lo standard di fatto.

CONCETTI TEORICI DA PADRONEGGIARE

- Perché esistono le code: disaccoppiamento temporale e strutturale, assorbimento dei picchi, resilienza, scalabilità indipendente dei consumer.
- Modelli: point-to-point contro publish/subscribe; code contro topic e subscription; competing consumer.
- **Garanzie di consegna:** at-most-once, at-least-once, exactly-once (e perché l'ultima è in pratica un'illusione); **idempotenza** e come ottenerla; ordinamento dei messaggi e sessioni.

- **Gestione dei fallimenti:** retry con backoff esponenziale, dead letter queue, poison message, circuit breaker.
- Tecnologie: **Azure Service Bus** o **RabbitMQ** (sceglierne una; i concetti trasferiscono); cenni a Kafka e in cosa un log distribuito differisce da una coda.
- **MQTT:** broker, topic, quality of service, retained message, last will. È il protocollo del mondo industriale e IoT.
- **Resilienza nelle chiamate:** retry, timeout, circuit breaker, bulkhead, fallback — e la libreria di riferimento in .NET (**Polly**, oggi integrata in `Microsoft.Extensions.Http.Resilience`). Vale sia per le code sia per ogni chiamata HTTP verso servizi esterni.
- Cenni a event-driven architecture, outbox pattern e saga; cenni ad astrazioni come MassTransit.

ESERCIZI

1. **Ingestione via coda.** Le letture non entrano più direttamente nel database: vengono pubblicate su una coda e consumate da un worker separato. *Obiettivo:* è esattamente il modo in cui un sistema industriale reale gestisce i picchi di dati dalle macchine.
2. **Prova di resilienza.** Spegni il worker, invia cento letture, riaccendilo: non deve perdersene nessuna. *Obiettivo:* dimostrare a te stesso il valore del disaccoppiamento.
3. **Idempotenza.** Fai consumare due volte lo stesso messaggio e rendi l'operazione idempotente. *Obiettivo:* at-least-once significa che questo *succederà*.
4. **Dead letter.** Configura retry con backoff e una dead letter queue, poi provoca un messaggio che ci finisce e gestiscilo.
5. **MQTT.** Simula una macchina che pubblica letture via MQTT verso un broker, con il sistema in ascolto. *Obiettivo:* collega direttamente la roadmap al dominio industriale in cui già lavori.

DOMANDE TIPICHE DA COLLOQUIO

• Perché useresti una coda invece di una chiamata HTTP diretta?
• Cosa significa at-least-once e quali conseguenze ha sul tuo codice?
• Cos'è l'idempotenza e come la implementi?
• A cosa serve una dead letter queue?
• Differenza tra una coda e Kafka.

> **Criterio di uscita:** l'esercizio 2 passa, e sai spiegare cosa accade al sistema se il consumer resta offline per un'ora.

Sezione 11bis — Microservizi, CQRS e architetture distribuite

Parte A teoria: 7 ore · CONSIGLIATO | Parte B laboratorio: 14 ore · COMPRIMIBILE

> **Perché questa sezione è divisa in due.** Le 7 ore coprono il *sapere*: il vocabolario, i compromessi, la capacità di rispondere e di argomentare a colloquio. È questa la parte che rende. Gli esercizi elencati più sotto — estrarre un servizio con database separato, gRPC, outbox, tracing end-to-end — sono **realisticamente 14 ore ulteriori**, non incluse nelle 7. La versione precedente di questo documento le sottostimava.
>
> **Parte A (teoria, consigliata):** i "Concetti teorici" e le "Domande da colloquio". Bastano per sostenere una conversazione tecnica credibile sull'argomento.
> **Parte B (laboratorio, comprimibile):** gli esercizi e l'incremento sul progetto. Serve se vuoi anche *mostrare* un sistema distribuito, non solo parlarne.

OBIETTIVO

"Microservizi" è probabilmente il termine più ricorrente negli annunci .NET di livello medio. Fino a qui la roadmap ne ha insegnato tutti i mattoni — API, code, container, tracing, database — senza mai nominarli come insieme. Questa sezione li ricompone e dà il vocabolario. Non per costruire microservizi: per **saperne parlare con cognizione e sapere quando non farli**, che è la risposta che colpisce di più a colloquio.

CONCETTI TEORICI DA PADRONEGGIARE

- **Monolite, monolite modulare, microservizi:** cosa si guadagna e cosa si paga davvero; il costo operativo nascosto; perché il consiglio prevalente è partire dal monolite modulare.
- **Confini: bounded context** dal Domain-Driven Design; come si individua un confine; il fallimento tipico è il "monolite distribuito", cioè servizi che non possono essere rilasciati indipendentemente.
- **Dati: database per servizio**; niente join tra servizi; duplicazione controllata; consistenza eventuale.
- **Comunicazione:** sincrona contro asincrona; **gRPC** (contratti, Protobuf, streaming) e quando conviene rispetto a REST; comunicazione a eventi e perché disaccoppia meglio.
- **Transazioni distribuite:** perché il two-phase commit non si usa; il pattern **saga** (coreografia contro orchestrazione); compensazione; il pattern **outbox** per non perdere eventi tra database e broker.
- **CQRS:** separare comandi e query; quando ha senso e quando è sovraingegneria; implementazione leggera con **MediatR**; cenni a event sourcing e perché è una cosa diversa e molto più impegnativa.
- **Attraversare i servizi: correlation id** e tracing distribuito (ricollegandosi a OpenTelemetry della Sezione 6bis); API gateway e BFF; service discovery; configurazione centralizzata.
- **Resilienza a livello di sistema:** timeout a cascata, circuit breaker tra servizi, degradazione elegante, idempotenza end-to-end.
- **Il rovescio della medaglia:** debugging distribuito, ambienti di sviluppo locali complessi, versionamento dei contratti, chi è di guardia quando qualcosa si rompe.

ESERCIZI — PARTE B, LABORATORIO (14 ORE) · COMPRIMIBILE

> **Se salti la Parte B, cosa perdi.** La capacità di *mostrare* un sistema distribuito, non quella di parlarne: la Parte A ti dà già le risposte da colloquio. Perdi anche il costo operativo provato sulla pelle, che è il vero insegnamento dei microservizi. Rimandabile a dopo l'assunzione senza danni.

1. **Estrazione di un servizio.** Estrai da MachineHub un singolo servizio autonomo — per esempio la gestione allarmi — con il proprio database, comunicante via eventi. *Obiettivo:* un microservizio estratto sul serio insegna più di dieci letti; e ti fa sentire di persona il costo operativo.
2. **Contratto gRPC.** Esponi un'operazione interna via gRPC accanto alla stessa via REST e confronta contratto, prestazioni ed ergonomia. *Obiettivo:* gRPC compare negli annunci e quasi nessun candidato l'ha mai toccato.
3. **Outbox.** Implementa il pattern outbox per garantire che un evento non si perda mai tra il salvataggio nel database e la pubblicazione sulla coda. Poi provoca deliberatamente un crash nel mezzo e verifica. *Obiettivo:* è il problema che tutti scoprono tardi e in produzione.
4. **CQRS leggero.** Introduci MediatR separando comandi e query su un'area del progetto. *Obiettivo:* vedere il beneficio e anche il costo in righe di codice, per poterne parlare con equilibrio.
5. **Tracciamento end-to-end.** Segui una singola richiesta attraverso API, evento, servizio allarmi e database, con un unico correlation id. *Obiettivo:* capire perché senza tracing distribuito i microservizi sono indebuggabili.

INCREMENTO SUL PROGETTO

MachineHub diventa un piccolo sistema distribuito: due servizi con database separati, comunicazione a eventi, outbox, correlation id propagato e tracing end-to-end. Nel `DECISIONS.md` va scritto anche perché per un sistema di queste dimensioni un monolite modulare sarebbe stata la scelta più sensata, e che questa estrazione è didattica.

DOMANDE TIPICHE DA COLLOQUIO

• Quando sceglieresti i microservizi e quando un monolite?

• Come decidi dove passa il confine tra due servizi?

- Come gestisci una transazione che coinvolge due servizi?
- Cos'è il pattern outbox e quale problema risolve?
- Cos'è CQRS? Serve sempre?
- gRPC o REST: come scegli?
- Come debug… come diagnostichi un problema che attraversa quattro servizi?
- Cos'è un monolite distribuito e perché è il peggiore dei mondi?

> **Criterio di uscita:** hai due servizi che comunicano a eventi con tracing end-to-end funzionante, e sai sostenere in colloquio *entrambe* le posizioni — perché adottare i microservizi e perché evitarli — con argomenti concreti.

MODULO E — Prova

Sezione 12 · 35 ore · Trasformare il lavoro fatto in materiale da colloquio.

Sezione 12 — Consolidamento, deploy e documentazione

35 ore · Modulo E · **CONSIGLIATO**

OBIETTIVO

Questa sezione vale quanto le altre, perché produce ciò che il selezionatore vede davvero. Un progetto eccellente ma non documentato non esiste.

ATTIVITÀ

1. **Refactoring con gli occhi di oggi.** Le prime sezioni le hai scritte quando sapevi meno. Rileggile e correggile. *Perché:* il refactoring del proprio codice vecchio è la prova più concreta che sei cresciuto, ed è un ottimo aneddoto da colloquio.
2. **README professionale:** problema risolto, diagramma dell'architettura, stack tecnologico, istruzioni di avvio locale in tre righe, link all'ambiente in cloud, screenshot o esempi di chiamata.
3. **Documentazione API** completa con Swagger/OpenAPI, con esempi di richiesta e risposta.
4. **Rifinitura dei tre artefatti anticipati** a fine Modulo B (README, deploy pubblico, Swagger): ora vanno portati a livello definitivo, non creati da zero.
5. `docs/security.md` completato: ogni misura di sicurezza adottata e la minaccia che neutralizza. *Perché:* pochissimi portfolio junior hanno un documento di sicurezza, e la sua sola esistenza sposta la percezione del candidato.
6. `docs/performance.md` completato: metodo di misura, numeri prima e dopo, collo di bottiglia individuato e intervento fatto.
7. `DECISIONS.md`**:** elenco delle scelte tecniche e delle relative motivazioni, incluse le alternative scartate. *Perché:* è il documento che ti salva a colloquio, perché dimostra che hai ragionato invece di copiare. È anche la risposta migliore all'obiezione "quanto di questo l'ha scritto l'AI".
8. **Diagrammi:** architettura dei componenti, schema del database, flusso di una lettura dalla macchina al report.
9. **Prova a voce:** registrati mentre spieghi il progetto in cinque minuti, in italiano e in inglese. Riascoltati e rifallo finché non scorre. *Perché:* saper fare non basta, a colloquio bisogna saper raccontare.
10. **I tre racconti da due minuti**, preparati e provati **in italiano e in inglese**. Sono la valuta del colloquio tecnico: alla domanda "raccontami un problema difficile che hai risolto" si risponde con questi, non improvvisando.
    a. **Un'ottimizzazione misurata:** qual era il problema, come l'hai misurato, cosa hai cambiato, i numeri prima e dopo. Materiale dalla Sezione 3 o 6bis.
    b. **Una vulnerabilità chiusa:** cosa hai trovato nel tuo stesso codice, come l'avresti sfruttata, come l'hai risolta. Materiale dalla Sezione 5bis.
    c. **Un problema di produzione diagnosticato:** il sintomo, gli strumenti usati, il percorso fino alla causa radice. Materiale dalla Sezione 6bis.

    Perché: ognuno dei tre dimostra una qualità diversa — rigore, responsabilità, metodo — e insieme coprono quasi tutte le domande comportamentali di natura tecnica.
11. **Ripasso mirato** delle sezioni con le note più deboli in `dotnet-lab`.
12. **Aggiornamento di CV e LinkedIn** usando il vocabolario degli annunci: non "sviluppo driver per macchinari" ma "integrazione ERP/MES, acquisizione dati di produzione, Industria 4.0, troubleshooting OT/IT".

> **Criterio di uscita:** una persona tecnica che non ti conosce apre il repository, capisce in cinque minuti cosa fa e com'è costruito, e tu sai difendere ogni scelta che ci trova dentro.

Tracker di avanzamento

#	Sezione	Ore	Mod.	Priorità	Iniziata	Completata
0	Setup e fondamenta	3	A	NUCLEO		
1	C#: dalle basi alla profondità	38	A	NUCLEO		
2	OOP, design e architettura	20	A	NUCLEO		
3	SQL e database relazionali	30	B	NUCLEO		
3bis	HTTP e fondamenta del web	4	B	NUCLEO		
4	ASP.NET Core	30	B	NUCLEO		
5	Entity Framework Core	20	B	NUCLEO		
5bis	Sicurezza applicativa	8	B	NUCLEO		

⚑ SOGLIA DI CANDIDABILITÀ — 153 ore di nucleo, ~4 mesi					**README + deploy pubblico + Swagger**
5ter	Real-time con SignalR	4	B	COMPRIM.	
6	Testing	15	C	NUCLEO	
6bis	Diagnostica e performance	6	C	CONSIGL.	
7	Git avanzato	8	C	NUCLEO	
7bis	Agile e Scrum	2	C	CONSIGL.	
8	Docker e container	15	D	NUCLEO	
9	CI/CD	12	D	NUCLEO	
10	Azure	35	D	NUCLEO	
10bis	NoSQL	3	D	COMPRIM.	
11	Messaggistica asincrona	12	D	CONSIGL.	
11bis A	Microservizi — teoria	7	D	CONSIGL.	
11bis B	Microservizi — laboratorio	14	D	COMPRIM.	
12	Consolidamento e deploy	35	E	CONSIGL.	
—	Appendice	2	—	COMPRIM.	

Regole permanenti

1. **L'AI può scrivere codice che sapresti scrivere. Mai codice che non sapresti scrivere.** Nel primo caso è un acceleratore, nel secondo è un prestito che non stai restituendo.
2. **Ogni riga che accetti, devi saperla spiegare.** Se non ci riesci, non è finita: quella riga è la tua prossima cosa da studiare.
3. **Digita, non incollare.**
4. **Una task a settimana scritta interamente a mano**, poi confrontata con la soluzione che avrebbe prodotto l'AI. La differenza è la tua lista di studio, generata automaticamente e gratis.
5. **Diario tecnico.** Ogni problema risolto: cosa succedeva, perché, come l'hai risolto. Dopo sei mesi è il tuo manuale personale, e la cura più efficace contro la sensazione di non sapere niente.
6. **Quando ti viene voglia di cambiare direzione** — e ti verrà — chiediti: questa cosa mi serve per il prossimo colloquio, o mi serve per sentirmi meno in colpa di non saperla? Se è la seconda, torna alla lista.
7. **Tutto su Linux dove possibile:** WSL per l'ambiente di sviluppo, container Linux, App Service su Linux. *Perché:* .NET moderno gira su Linux ovunque in produzione, e la familiarità con la riga di comando Linux è data per scontata in ogni contesto cloud e DevOps.
8. **Gli script di supporto si scrivono in Bash o Python**, non come applicazioni console C#. *Perché:* nessuno in un team scrive un progetto .NET per rinominare dei file, e saper mettere insieme uno script è una competenza attesa e separata dal saper programmare.
9. **La task settimanale scritta a mano si fa a cronometro e con il completamento automatico dell'IDE disattivato.** *Perché:* è la simulazione onesta di una prova tecnica. Scrivere a mano con IntelliSense attivo non misura la stessa cosa.
10. **MachineHub deve girare almeno una volta anche su PostgreSQL.** *Perché:* costa poche ore, dimostra che l'applicazione non è incollata a un fornitore, e raddoppia il numero di annunci in cui la tua esperienza sui database è spendibile.

Appendice — Consapevolezza tecnologica

2 ore · da leggere, non da studiare · `COMPRIMIBILE`

> **Se la salti, cosa perdi.** Solo la capacità di riconoscere dei nomi. Ma costa due ore di lettura e non richiede esercizi: è il rapporto costo/beneficio migliore del documento, ed è comprimibile solo perché non è una competenza.

> **Come si usa questa appendice.** Nessuna di queste tecnologie va imparata adesso. L'obiettivo è diverso e più economico: **sapere che esistono, che problema risolvono e quando qualcuno le nominerebbe**. Due ore di lettura, una volta sola. Serve a non restare muti quando in un colloquio o in una riunione salta fuori un nome che non hai mai sentito — situazione che fa perdere più punti di quanti ne farebbe guadagnare conoscerle a fondo.

GRPC

Framework di comunicazione ad alte prestazioni basato su contratti Protobuf e HTTP/2. Nell'ecosistema Microsoft ha di fatto **preso il posto di WCF** per la comunicazione tra servizi: se in un annuncio o in un colloquio senti nominare WCF su un progetto nuovo, gRPC è la risposta moderna. Approfondito operativamente nella Sezione 11bis.

HANGFIRE E QUARTZ.NET

Librerie per lavori pianificati e in background persistenti: job ricorrenti, ritardati, con ritentativi e cruscotto di monitoraggio. Rispondono alla domanda "e se il processo si riavvia mentre il job è a metà?", a cui un semplice `BackgroundService` non risponde. Compaiono spesso negli annunci enterprise.

OPENTELEMETRY

Standard aperto e indipendente dal fornitore per log, metriche e tracce. È la direzione verso cui converge tutta l'industria dell'osservabilità. Usato praticamente nella Sezione 6bis; qui basta sapere che la sua adozione è ciò che ti permette di cambiare backend di monitoraggio senza riscrivere la strumentazione.

MASSTRANSIT E NSERVICEBUS

Astrazioni sopra i broker di messaggi (RabbitMQ, Azure Service Bus, Kafka) che forniscono già pronti retry, dead letter, saga, outbox e serializzazione. Nella Sezione 11 quei meccanismi si implementano a mano *per capirli*; in azienda si usa quasi sempre una di queste due.

BLAZOR

Framework per scrivere interfacce web in C# invece che in JavaScript, nelle varianti Server e WebAssembly. È l'estensione naturale se un domani servisse il front-end senza cambiare linguaggio, e compare regolarmente tra le competenze preferite negli annunci .NET.

MULTI-TENANCY

Come un'unica applicazione serve più clienti tenendone separati i dati: database per tenant, schema per tenant, o discriminatore di colonna — con i rispettivi compromessi su costi, isolamento e complessità. È un requisito ricorrente in ogni prodotto SaaS e una domanda di design frequente.

FEATURE FLAG

Attivare e disattivare funzionalità a runtime senza rilasciare nuovo codice (LaunchDarkly, Azure App Configuration). Rendono possibili rilasci progressivi, test A/B e disattivazione immediata di una funzione problematica. Sono il presupposto pratico del continuous deployment.

MIGRAZIONE DA .NET FRAMEWORK A .NET MODERNO

Una quota consistente del lavoro .NET reale è manutenzione ed evoluzione di applicazioni su .NET Framework. Vale la pena conoscere l'esistenza del percorso di migrazione, degli strumenti di assistenza, dell'approccio incrementale con lo strangler fig pattern, e dei blocchi tipici (WCF, Web Forms, dipendenze solo Windows). *È anche il rovescio di un vantaggio:* chi sa muoversi in un codebase legacy è un profilo scarso sul mercato e sottovalutato negli annunci.

Dopo questa roadmap

Al completamento, le direzioni naturali sono tre. Vanno affrontate una alla volta, con una roadmap dedicata ciascuna, e la scelta si farà con dati reali in mano invece che con ipotesi:

- **Approfondimento backend/architettura:** sistemi distribuiti, performance, design di sistema, event-driven architecture.
- **DevOps e platform engineering:** Kubernetes, Terraform e infrastructure as code, observability, SRE.
- **Cloud security e DevSecOps:** il percorso che integra sicurezza e delivery, per cui le fondamenta poste nella sezione 10 (identità, segreti, privilegio minimo) sono il primo mattone.

Le stime orarie presuppongono circa un anno di esperienza pratica pregressa in C#/.NET e SQL. Codici e prezzi delle certificazioni vanno verificati sui siti ufficiali, perché cambiano. Rivedere e ricalibrare questo documento al completamento della sezione 6.